Red Hat OpenShift AI Self-Managed 3.4

Release notes

Features, enhancements, resolved issues, and known issues associated with this release

Last Updated: 2026-05-15

Red Hat OpenShift AI Self-Managed 3.4 Release notes

Features, enhancements, resolved issues, and known issues associated with this release

Legal Notice

Abstract

These release notes provide an overview of new features, enhancements, resolved issues, and known issues in version 3.4 of Red Hat OpenShift AI.

Table of Contents

CHAPTER 1. OVERVIEW OF OPENSHIFT AI

Red Hat OpenShift AI is a platform for data scientists and developers of artificial intelligence and machine learning (AI/ML) applications.

OpenShift AI provides an environment to develop, train, serve, test, and monitor AI/ML models and applications on-premise or in the cloud.

For data scientists, OpenShift AI includes Jupyter and a collection of default workbench images optimized with the tools and libraries required for model development, and the TensorFlow and PyTorch frameworks. Deploy and host your models, integrate models into external applications, and export models to host them in any hybrid cloud environment. You can enhance your projects on OpenShift AI by building portable machine learning (ML) workflows with AI pipelines by using Docker containers. You can also accelerate your data science experiments through the use of graphics processing units (GPUs) and Intel Gaudi AI accelerators.

For administrators, OpenShift AI enables data science workloads in an existing Red Hat OpenShift or ROSA environment. Manage users with your existing OpenShift identity provider, and manage the resources available to workbenches to ensure data scientists have what they require to create, train, and host models. Use accelerators to reduce costs and allow your data scientists to enhance the performance of their end-to-end data science workflows using graphics processing units (GPUs) and Intel Gaudi AI accelerators.

OpenShift AI has a **Self-managed software** deployment option that you can install on-premise or in the cloud. You can install OpenShift AI Self-Managed in a self-managed environment such as OpenShift Container Platform, or in Red Hat-managed cloud environments such as Red Hat OpenShift Dedicated (with a Customer Cloud Subscription for AWS or GCP), Red Hat OpenShift Service on Amazon Web Services (ROSA classic or ROSA HCP), or Microsoft Azure Red Hat OpenShift.

For information about OpenShift AI supported software platforms, components, and dependencies, see the Supported Configurations for 3.x Knowledgebase article.

For a detailed view of the 3.4 release lifecycle, including the full support phase window, see the Red Hat OpenShift AI Self-Managed Life Cycle Knowledgebase article.

CHAPTER 2. NEW FEATURES AND ENHANCEMENTS

This section describes new features and enhancements in Red Hat OpenShift AI 3.4 GA.

2.1. NEW FEATURES

2.1.1. 3.4 GA new features

Migration guide available for transitioning from vLLM-based InferenceService to LLMInferenceService

Platform Operators deploying Distributed Inference with llm-d on OpenShift AI can follow a step-by-step guide covering LLMInferenceServiceConfig, YAML examples, and migration from vLLM-based InferenceService deployments. The guide explains how LLMInferenceServiceConfig replaces custom ServingRuntime definitions. For more information, see https://access.redhat.com/articles/7141739.

Prometheus metrics for Distributed Inference with llm-d

You can now monitor llm-d distributed inference deployments using documented Prometheus metrics and PromQL query examples. These metrics are exported by all llm-d components, including the Endpoint Picker (EPP), vLLM engine pods, and prefix cache, along with example queries for building custom dashboards and configuring ServiceMonitors for OpenShift User Workload Monitoring.

MLFlow SDK pre-installed in workbench and runtime images

The MLFlow SDK is now pre-installed and included in the datascience, tensorflow (CUDA & ROCm), pytorch (CUDA & ROCm), and codeserver workbench and runtime images.

MLflow Operator is now a managed component in the DataScienceCluster CR

Starting with Red Hat OpenShift AI 3.4, the MLflow Operator is a managed component in the **DataScienceCluster** custom resource (CR). You can enable the **mlflowoperator** component by setting **managementState** to **Managed** in the **DataScienceCluster** CR. MLflow availability in the dashboard is now determined by the component state, and the deprecated **mlflow** dashboard feature flag is no longer required. For more information, see Enable the MLflow operator component.

NeMo Guardrails to enable AI safety

NeMo Guardrails, introduced in Red Hat OpenShift AI 3.3 as a Technology Preview, is fully supported with this release. You can add guardrails and safety controls to your deployed models. NeMo Guardrails provides a framework for controlling conversations with large language models, enabling you to define a variety of rails, such as sensitive data detection, content filtering, or custom validation rules. NeMo Guardrails introduces the following capabilities:

- **/v1/guardrails/checks** endpoint for standalone querying of guardrail policies
- Full OpenAI compatibility, with a new **/v1/models/** endpoint
- New regex rails for regex-based guardrail logic
- Support for multiple replicas of the NeMo Guardrails pod to improve scalability
- Out-of-the-box OpenTelemetry support
- Automatic redeployment after configuration changes, for zero-downtime guardrail tuning

Models-as-a-Service now Generally Available

Models-as-a-Service (MaaS) is now Generally Available in Red Hat OpenShift AI 3.4. Previously introduced as a Technology Preview feature in version 3.3, MaaS provides an enterprise platform for centralized governance, consumption tracking, and self-service access to large language models. MaaS addresses resource consumption and governance challenges by exposing models through managed API endpoints with subscription-based controls. Administrators can define subscriptions that grant groups access to specific models with configurable token limits, while users can generate and manage their own API keys for programmatic access.

Key capabilities include:

- Subscription-based model access with configurable token quotas and rate limiting
- Self-service API key generation and management
- Centralized authentication and authorization policies (external OIDC authentication available as Technology Preview)
- Support for distributed inference with llm-d (vLLM runtime support available as Technology Preview)
- Usage tracking and showback reporting through an observability dashboard (Technology Preview)
- Routing to external model providers such as OpenAI or Anthropic (Technology Preview)

For more information, see Governing LLM access with Models-as-a-Service .

The Models-as-a-Service subscription model redesign

The Models-as-a-Service (MaaS) subscription model has been redesigned to replace the tier-based model introduced in version 3.3. The new subscription model provides administrators with flexible access control and resource management for large language models, while enabling data scientists to select from available subscriptions when accessing models.

Key enhancements include:

- Priority-based subscription assignment: As a cluster administrator, you can assign priority levels to subscriptions. As a user, when you belong to multiple groups with different subscriptions, the system automatically assigns you to the highest-priority subscription by default, while allowing you to manually select from your available subscriptions when generating API keys.
- Group-based access control: As a cluster administrator, you can define subscriptions that grant groups access to specific models. Subscriptions support integration with OpenShift groups, OIDC group claims, and API key group snapshots.
- Configurable token quotas: As a cluster administrator, you can define distinct token limits per model for each subscription, enabling cost control and resource allocation aligned with organizational policies. As a user, your token consumption is tracked against your active subscription's limits.
- Authorization policy integration: Subscriptions work in combination with authorization policies to control API gateway access and token consumption limits.

The subscription model redesign enables organizations to enforce consumption policies across teams while maintaining self-service access for data scientists and developers.

For more information, see Governing LLM access with Models-as-a-Service .

Self-service API key management for Models-as-a-Service

You can now create and manage your own API keys for programmatic access to large language models through Models-as-a-Service. This self-service capability streamlines access to large language models while maintaining centralized governance through subscriptions and authorization policies.
Key capabilities include:

- User-managed API key lifecycle: Create permanent API keys with configurable expiration dates, view active keys, and revoke keys when no longer needed
- Temporary API key generation: Generate short-lived API keys directly from the model endpoint dialog for quick testing and prototyping
- Subscription-scoped authentication: API keys are scoped to specific subscriptions, inheriting the subscription's model access and token limits
- Group membership snapshot: User group membership is captured at API key creation time, ensuring consistent access control even when group assignments change

Self-service API key management empowers you to access models programmatically through OpenAI-compatible APIs without administrative bottlenecks, while administrators retain control through subscription-based governance.

For more information, see Governing LLM access with Models-as-a-Service .

Support for Llama Stack and KubeRay on IBM Power

Red Hat OpenShift AI 3.4 GA introduces official support for both Llama Stack and KubeRay on the IBM Power architecture.

OCI-compliant storage layer for model registry

You can now use the OpenShift AI dashboard to register a model from an S3-compatible source or URI, transform it into an OCI ModelCar image, and store it in an OCI registry. The ModelCar target format enables fast deployment with KServe.
The model transfer job runs as a background Kubernetes Job that you can monitor from the dashboard. This feature provides the following capabilities:

- Register and store models from object storage (S3, MinIO) or URLs in a single operation.
- Models are automatically converted to ModelCar OCI images, ensuring compatibility with KServe ModelCar for model serving.
- Track model transfer jobs in real-time with detailed status information, Kubernetes event logs, and automatic polling.
- Retry failed jobs or delete completed jobs directly from the dashboard.
- ConfigMaps and Secrets are automatically garbage-collected when jobs are deleted.

MLServer ServingRuntime for KServe is now generally available

The **MLServer ServingRuntime for KServe** is now generally available in Red Hat OpenShift AI. You can use this runtime to deploy models trained on structured data, such as classical machine learning models. You can deploy models directly in their native format, simplifying the deployment process.

Supported model frameworks include: * Scikit-learn * XGBoost * LightGBM * ONNX

For models with well-known file names, MLServer automatically configures all required environment variables during deployment through the **Deploy a model** wizard.

For more information, see Deploying models using the MLServer runtime .

2.1.2. 3.4 EA2 new features

Multi-architecture support for the model catalog

The model catalog includes support for **IBM Power (ppc64le)** architecture. With this enhancement, you can discover and deploy models directly from the dashboard. Support is available for the following validated models:

- **registry.redhat.io/rhai/modelcar-granite-3-3-8b-instruct**
- **registry.redhat.io/rhai/modelcar-granite-4-0-h-small:3.0**
- **registry.redhat.io/rhai/modelcar-granite-4-0-h-tiny:3.0**

Multi-architecture support for the model catalog

The model catalog now includes support for the IBM Z architecture. This enhancement enables users on IBM Z platforms to discover and deploy models directly from the OpenShift AI dashboard. Currently, support is available for the following model:

- **registry.redhat.io/rhai/modelcar-granite-3-3-8b-instruct**

Just-In-Time Checkpointing and S3 Storage for Kubeflow Trainer

Kubeflow Trainer now provides Just-In-Time (JIT) and periodic checkpointing for distributed training jobs on OpenShift AI. This enhancement automatically saves the training state, including model weights, optimizer state, and training step at regular intervals and immediately before interruptions such as preemption, eviction, or maintenance. Interrupted jobs automatically resume from the latest valid checkpoint, significantly reducing wasted GPU compute and improving overall training efficiency.
Checkpoints can be stored on PersistentVolumeClaims (PVCs) or S3-compatible object storage. With S3, checkpoints are uploaded in the background without pausing training, enabling low-overhead, continuous protection of progress. S3-backed storage also provides a cost-efficient, portable alternative to PVCs, allowing checkpoints to be retained, shared, and reused across clusters.

2.1.3. 3.4 EA1 new features

Model deployments are not visible under the model registry deployments tab on IBM Power (ppc64le) in RHOAI 3.4-EA1.

Workbench and runtime images default to Red Hat Python index

Workbench and runtime images default to the Red Hat Python index. When you install or update Python packages, packages are pulled from the Red Hat Python index rather than PyPI. This provides you with Red Hat built and supported Python packages.

Garak evaluation provider available in Llama Stack distribution

The Garak evaluation provider is available in the Llama Stack distribution. Garak provides security

scanning capabilities for large language models to help identify potential vulnerabilities and safety issues. The provider is available in two versions: an inline version that runs scans in the same process as the Llama Stack server, and a remote version that runs scans by using Kubeflow Pipelines.

PostgreSQL database support for Model Registry

You can configure a PostgreSQL database as the backend for Model Registry from the OpenShift AI dashboard.

Default database solution for Model Registry

Model Registry includes a default database solution for testing. Use this solution to start using Model Registry without configuring an external database.

NOTE

The default database is not intended for production workloads.

2.2. ENHANCEMENTS

2.2.1. 3.4 GA enhancements

vLLM uvicorn access logs are disabled by default in Distributed Inference with llm-d

vLLM uvicorn access logs are disabled by default in LLMInferenceServiceConfig, including logs generated by router-scheduler /metrics endpoint polling. This reduces excessive logging caused by the EndpointPicker scraping metrics scraping every 200 milliseconds. Operators who need access logs for debugging can re-enable them explicitly.

2.2.2. 3.4 EA2 enhancements

Simplified configuration for Distributed Inference with llm-d scheduler settings

Configure Distributed Inference with llm-d scheduler settings using the **endpointPickerConfig** field in the LLMInferenceService specification. You can specify the configuration inline or reference a ConfigMap. This approach replaces the previous method that required making extensive specifications in the EndpointPicker configuration in the scheduler's **--configText** argument.

Configure vLLM runtime arguments using Kubernetes container args field

You can configure vLLM runtime arguments using the standard Kubernetes container **args** field in LLMInferenceService resources. User-specified arguments are merged with system defaults, allowing you to add new arguments or override specific defaults without replacing the entire argument list.

The previous **VLLM_ADDITIONAL_ARGS** environment variable method continues to work for backward compatibility.

2.2.3. 3.4 EA1 enhancements

Hybrid search support for Qdrant remote vector database provider

Vector Store Search supports hybrid and keyword search for the Qdrant Vector IO provider.

CHAPTER 3. TECHNOLOGY PREVIEW FEATURES

IMPORTANT

This section describes Technology Preview features in Red Hat OpenShift AI 3.4 GA. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope .

3.1. 3.4 GA TECHNOLOGY PREVIEW FEATURES

Workload variant autoscaler for Distributed Inference with llm-d model deployments

Platform Operators can enable autoscaling for Distributed Inference with llm-d model deployments based on incoming request volume as a Technical Preview. Scaling decisions use inference request metrics as the signal, so replicas respond to actual demand.

Priority-based flow control for mixed llm-d workloads

Platform Operators can assign priority tiers to workload classes and configure queuing policies so that latency-sensitive requests are served ahead of throughput-oriented batch traffic. Multiple workload profiles can share a single distributed inference with llm-d deployment without dedicated GPU pools per priority level.

Gateway discovery for llm-d deployments

You can discover and select an existing Kubernetes Gateway resource from the model serving UI when deploying LLMInferenceService models. This Technology Preview feature enables self-service Gateway management, supports multitenant namespace-scoped network isolation, and provides programmatic access through the Gateway discovery REST API.
Gateway discovery and creation is disabled by default and must be explicitly enabled by a cluster administrator through the dashboard configuration.

vLLM runtime support for Models-as-a-Service

You can now deploy models using the vLLM runtime through Models-as-a-Service (MaaS). This Technology Preview feature enables you to serve large language models with vLLM's high-performance inference capabilities while benefiting from MaaS governance and subscription-based controls.
Key capabilities include:

- vLLM deployment from the dashboard: As a cluster administrator, you can enable vLLM runtime support for MaaS by setting the **vLLMDeploymentOnMaaS** feature flag in the **OdhDashboardConfig** custom resource. Once enabled, you can deploy models using vLLM runtime directly from the Models-as-a-Service interface.
- Performance optimization: vLLM provides optimized inference performance for large language models through features such as continuous batching, PagedAttention, and efficient memory management, improving throughput and reducing latency for model serving workloads.

- Subscription-based governance: As a cluster administrator, you can apply the same subscription-based access controls, token quotas, and authorization policies to vLLM-served models as you do for other MaaS models, maintaining consistent governance across different serving runtimes.
- Unified user experience: As a user, you access vLLM-served models through the same OpenAI-compatible API endpoints and authentication methods as other MaaS models, providing a seamless experience regardless of the underlying serving runtime.

vLLM runtime support for Models-as-a-Service enables organizations to leverage high-performance inference capabilities while maintaining centralized governance and cost control.

For more information, see Governing LLM access with Models-as-a-Service .

External OIDC authentication for Models-as-a-Service

You can now configure Models-as-a-Service to authenticate users with an external OpenID Connect (OIDC) identity provider. This Technology Preview feature enables enterprise-wide access to large language models without requiring OpenShift accounts for every user.
Key capabilities include:

- External identity provider integration: As a cluster administrator, you can integrate MaaS with existing identity providers, leveraging your organization's existing authentication infrastructure.
- OIDC group mapping: As a cluster administrator, you can map external user groups from OIDC group claims to MaaS subscriptions, enabling centralized access control and quota enforcement based on your organization's existing group structure.
- Seamless user experience: As a user, you authenticate using your existing enterprise credentials and receive API keys scoped to your OIDC group memberships, providing consistent access across your organization's AI platform.
- Multiple authentication methods: MaaS supports authentication through OpenShift groups and OIDC token claims, allowing you to choose the authentication method that best fits your deployment model.

External OIDC authentication enables organizations to integrate MaaS into existing identity and access management workflows while maintaining self-service access for data scientists and developers.

For more information, see Governing LLM access with Models-as-a-Service .

Models-as-a-Service observability dashboard

As a cluster administrator, you can now monitor platform-wide Models-as-a-Service (MaaS) usage through a dedicated observability dashboard embedded in the OpenShift AI console. This Technology Preview feature provides comprehensive usage metrics for cost attribution and showback reporting to finance teams.
Key capabilities include:

- Subscription-level metrics: View total tokens consumed, total requests, error counts, and success rates across all subscriptions, or drill down to specific subscriptions for detailed analysis

- Token consumption tracking: Track token usage by user, subscription, and model, with detailed tables showing request counts and rate limit violations for cost allocation and capacity planning
- Filtering and time ranges: Filter metrics by user, subscription, and model to analyze specific usage patterns. View metrics for configurable time periods ranging from the last 5 minutes to the last 14 days, or specify custom date ranges
- CSV export: Export usage data in CSV format for integration with external metering, billing, and financial reporting systems
- Prometheus metrics integration: The dashboard queries Prometheus metrics collected from Kuadrant and MaaS components, providing real-time visibility into platform health and resource consumption

For more information, see Governing LLM access with Models-as-a-Service .

External model egress via inference gateway

You can now route model inference requests to external model providers through the Models-as-a-Service (MaaS) inference gateway. This Technology Preview feature enables you to apply MaaS governance policies, token tracking, and rate limiting to third-party LLM services such as OpenAI, Anthropic, or other external providers.
Key capabilities include:

- External model configuration: As a cluster administrator, you can define external LLM provider configurations using the ExternalModel custom resource, specifying the provider endpoint, authentication method, and model identifiers.
- Unified governance: As a cluster administrator, you can apply the same subscription-based access controls, authorization policies, and token quotas to external models as you do for models served within the cluster.
- Centralized usage tracking: As a cluster administrator, you can monitor token consumption and request metrics for external models alongside internally-served models in the observability dashboard, providing a unified view of LLM usage across your organization.
- Consistent API interface: As a user, you access external models through the same OpenAI-compatible API endpoints and authentication methods as internally-served models, providing a seamless experience regardless of where models are hosted.

External model egress enables organizations to maintain centralized governance and cost visibility when using external LLM providers while preserving flexibility for teams to leverage best-in-class models from multiple sources.

For more information, see Governing LLM access with Models-as-a-Service .

Llama Stack Responses API parity with OpenAI

The Llama Stack Responses API in OpenShift AI 3.4 introduces systematic alignment with OpenAI's Responses API. Previously, the Llama Stack Responses API diverged from OpenAI's Responses API specifications. This update allows you to use OpenAI compatible SDKs and clients with Llama Stack without error and removes differences in specifications including: field names, unsupported parameters, and response shapes. This creates a consistent experience of API specifications between Llama Stack and OpenAI.

Responses API on Llama Stack

The Responses API on Llama Stack is now available on OpenShift AI as a Technology Preview, previously available as a Developer Preview. This allows for a high-level orchestration layer for building agentic applications and provides closer OpenAI-compatibility in Llama Stack.

Automate machine learning model training with AutoML

AutoML is available as a Technology Preview feature in Red Hat OpenShift AI 3.4. You can use AutoML to train and compare machine learning models for your data. AutoML provides a dashboard UI to configure optimization runs, compare models on a leaderboard, generate notebooks for running models, and register a model for deployment. For more information, see Working with AutoML.

Automate RAG optimization with AutoRAG

AutoRAG is available as a Technology Preview feature in Red Hat OpenShift AI 3.4. You can use AutoRAG to find the best retrieval-augmented generation (RAG) configuration for your documents and use case. AutoRAG provides a dashboard UI to configure optimization runs, evaluate RAG patterns on a leaderboard, and generate notebooks to run patterns. For more information, see Working with AutoRAG.

Recommended vLLM runtime configurations in model catalog

Red Hat OpenShift AI now provides recommended vLLM runtime configurations for select high-demand models. These configurations are designed to help users achieve maximum performance on specific hardware profiles, matching or exceeding industry benchmarks for low-latency and high-throughput serving.
Previously, users had to manually tune vLLM arguments to find the sweet spot for performance. Now, validated YAML templates are embedded directly in the model card Readme section as markdown, serving as a reference for manual configuration.

Key features of this update include:

- Optimal performance recipes: Validated configurations optimized for specific hardware (initially NVIDIA H200) and workload profiles (8K prefill / 1K generation).
- Structured YAML metadata: Recommended settings include specific environment variables and CLI arguments verified by the Red Hat Performance and Scale (PSAP) team.
- Model card integration: No UI changes are required. Optimized recipes are appended to the model's Readme field for easy discovery.
- Performance parity: These configurations ensure OpenShift AI performance remains competitive with alternative serving stacks by leveraging the latest vLLM optimizations.

Initial targeted models:

- GPT-OSS 120B: Optimized for major enterprise usage.
- Llama 3-70B: Validated for standard high-performance deployments.
- DeepSeek R1: Provided as a showcase for advanced reasoning and Red Hat Summit demonstrations.

To find the optimal runtime settings for a supported model:

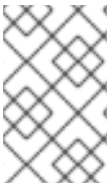
1. In the OpenShift AI dashboard, navigate to **AI hub → Catalog**.
2. Select one of the supported models, for example, GPT-OSS 120B.

3. On the model card, scroll to the readme section.
4. Locate the Recommended Configurations YAML block.

During this Technical Preview phase, you can manually apply these recommendations during the model deployment workflow:

1. Copy the CLI arguments and environment variables from the model card YAML template.
2. Initiate the Deploy workflow for the model.
3. In the Serving Runtime configuration, add the copied CLI arguments to the Additional Service Arguments field.
4. Add the recommended environment variables to the deployment configuration.

Technical details and scope: * Hardware profile: The initial set of recipes is specifically tuned for NVIDIA H200 GPUs. * Workload profile: Configurations are optimized for Online Serving (balancing latency and throughput) with a single profile of 8K prefill / 1K generation. * Validation: Every configuration has undergone accuracy and performance testing by the Red Hat Model Validation and PSAP teams.



NOTE

This Technical Preview feature is intended for feedback collection. Future releases will include UI enhancements such as one-click application of these presets and automated hardware detection.

Artifact signing and verification for model registry

Model registry now provides cryptographic signing and verification of AI artifacts by using the Python client API. This Technology Preview feature enables you to sign artifacts to establish authenticity and verify signatures to confirm integrity. Artifact signing provides foundational capabilities for model serving verification and AI asset provenance tracking in future releases.
For more information, see the Kubeflow Model Registry Python client documentation and Kubeflow async upload job documentation.

EvalHub client SDK and CLI

OpenShift AI includes a Technology Preview of the EvalHub client SDK and command-line interface (CLI). The EvalHub SDK provides a Python client library and CLI for integrating evaluation workflows into development environments, automation pipelines, and notebook-based experimentation.

This Technology Preview introduces the following capabilities:

- Python client library (eval-hub-sdk) for programmatic interaction with the EvalHub REST API, including job submission, provider discovery, and result retrieval
- CLI utilities through the evalhub command for submitting evaluation jobs, checking job status, retrieving results, and managing evaluation collections from the terminal
- Support for both synchronous and asynchronous evaluation workflows
- Authentication support for API keys and service accounts
- Adapter SDK for writing custom evaluation framework adapters

Install the client library with the **pip install "eval-hub-sdk[client]"** command, to include CLI support with **pip install"eval-hub-sdk[cli]"** and use "eval-hub-sdk" for installing only the minimum required for writing an EvalHub adapter using the Adapter SDK.

Evaluation Stack user interface

Red Hat OpenShift AI includes a Technology Preview of the Evaluation Stack user interface. The Evaluation Stack UI provides a guided workflow in the OpenShift AI dashboard for running model evaluations without requiring command-line tools or notebook workflows.

This Technology Preview introduces the following capabilities:

- Browse and select evaluation tasks from registered providers, including LM Evaluation Harness, RAGAS, Garak, and GuideLLM
- Run built-in evaluation collections that group benchmarks across providers into reusable suites
- Configure evaluation parameters such as sample count and maximum tokens per response
- Evaluate models or AI applications against standardized benchmarks or custom evaluation datasets
- Monitor evaluation progress in real time and cancel running evaluations
- View summarized evaluation scores and metrics

The Evaluation Stack UI requires EvalHub to be deployed on the cluster. For information about deploying and configuring EvalHub, see Evaluate LLMs with EvalHub.

3.2. 3.4 EA2 TECHNOLOGY PREVIEW FEATURES

Create ability to sign and verify AI Artifacts in Registry

For more information about signing and verifying models in the Model Registry see https://github.com/kubeflow/model-registry/tree/main/clients/python#signing-and-verifying-models. For async upload job documentation, see https://github.com/kubeflow/model-registry/tree/main/jobs/async-upload#signing

TLS and proxy configuration for all Llama Stack remote inference providers

Red Hat OpenShift AI 3.4 EA2 introduces a standardized network configuration block for all Llama Stack remote inference providers. This generalizes TLS and proxy settings, previously exclusive to the **remote::vllm** provider, to ensure compatibility across private network architectures.

Llama Stack versions in Red Hat OpenShift AI 3.4 EA2

Red Hat OpenShift AI 3.4 EA2 includes Open Data Hub Llama Stack version 0.6.0.1+rhai0, which is based on upstream Llama Stack version 0.6.0.

MLflow integration

Red Hat OpenShift AI includes a Technology Preview of MLflow. MLflow uses Kubernetes namespaces (OpenShift projects) as workspaces to provide logical isolation of experiments, registered models, and prompts. MLflow uses Kubernetes role-based access control (RBAC) to authorize API requests. For more information about enabling and using MLflow in OpenShift AI, see the Configuring MLflow in Red Hat OpenShift AI (Technology Preview) Knowledgebase article.

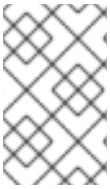
Support for text embedding models in the Model Catalog

Red Hat OpenShift AI includes a dedicated suite of text embedding models within the Model Catalog

that can be deployed on vLLM. This Technology Preview feature allows data scientists and AI engineers to discover and deploy models designed specifically for vector generation, a critical component for Retrieval-Augmented Generation (RAG) and semantic search workflows. By separating these models from standard generative large language models (LLMs), OpenShift AI provides a streamlined experience for users building vector-generation engines.
Key features of this update include:

- **Categorized discovery**: Embedding models are available under the **Other Models** category in the Model Catalog.
- **Distinct labeling**: Each model card is labeled with a **text-embedding** tag to clarify the model's function (outputting numerical vectors) compared to text-generation models.
- **Ready-to-deploy images**: The models are provided as OCI-compliant ModelCar images, ensuring they are ready for immediate instantiation as running services on an OpenShift cluster.
  Supported models include:
- Granite Embedding English R2
- Embedding Gemma 300M
- Nomic Embed Text v1.5
- Qwen3 Embedding 8B
- All-MiniLM-L6-v2

To view embedding models in the Catalog, navigate to **AI hub → Models → Catalog** in the OpenShift AI dashboard, select the **Other models** category, and look for the **text-embedding** tag on model cards. You can deploy a model directly from its card by following the deployment wizard to configure the serving runtime and hardware profile. Once deployed, the model provides an API endpoint that generates numerical embeddings for text inputs for use in downstream vector databases or RAG pipelines.



NOTE

Some models, such as Nomic Embed Text v1.5, require extra vLLM parameters to be set, such as **--trust-remote-code**. Refer to upstream Hugging Face model cards for specific details.

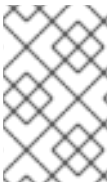
Workbench and runtime images default to the Red Hat Python index

Access and use Red Hat built and supported Python packages. Installing or updating Python packages will pull from the Red Hat Python index rather than PyPI.

YAML viewer for Distributed Inference with llm-d model deployments

The model deployment wizard includes a YAML viewer that provides real-time YAML preview and manual editing capabilities for LLMInferenceService deployments. You can toggle between Form and YAML views to see the generated LLMInferenceService YAML as you fill out the deployment form. The preview updates automatically as form fields change, enabling you to verify configuration before deployment. You can also enter Manual Edit Mode to edit YAML directly for advanced Distributed Inference with llm-d parameters not exposed in the form, such as autoscaling, disaggregated serving, LoRA adapters, and custom runtime configurations.
This Technology Preview feature introduces the following capabilities:

- Real-time YAML preview with automatic form-to-YAML synchronization
- Copy to clipboard and download functionality
- Manual Edit Mode for direct YAML editing of advanced configurations
- Automatic fallback to YAML editing when the wizard cannot parse deployment configurations (for example, deployments created via CLI or GitOps)
- Client-side YAML syntax validation before deployment



NOTE

Manual Edit Mode is a one-way operation. Once you enter manual YAML editing mode, you cannot return to the guided form view for that deployment session. Changes made in YAML edit mode bypass form validation.

3.3. 3.4 EA1 TECHNOLOGY PREVIEW FEATURES

Llama Stack versions in OpenShift AI 3.4 EA1

OpenShift AI 3.4 EA1 includes Open Data Hub Llama Stack version 0.5.0+rhai0, which is based on upstream Llama Stack version 0.5.0.

Gen AI Playground interface redesign

The Gen AI Playground interface provides a prompt-lab-style experience with improved prompt-driven experimentation, rapid iteration capabilities, and clear visual feedback. This redesign aligns the Playground experience with industry-standard patterns for GenAI tools.

Multi-instance chat comparison in Playground

You can compare results across multiple configurations in the Playground by using multiple chat panes side-by-side. Each pane supports unique configurations for models, prompts, MCP servers, guardrails, and knowledge sources. You can run synchronized prompts across all panes or use independent prompts for each pane.
Key capabilities include:

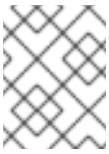
- Side-by-side output comparison
- Toggle individual chat panes for A/B-style testing
- Runtime information display including latency and token counts

Basic guardrails available in Gen AI Playground

The Gen AI Playground provides access to basic safety guardrails from Llama Stack. You can enable or disable guardrails in the playground interface to filter unsafe content and detect prompt injection attempts.
The following guardrails are available:

- Content Safety (Llama Guard): Filters categories such as hate, violence, sexual content, self-harm, criminal activity, and privacy violations.
- Prompt Injection / Jailbreak Detection (Prompt Guard): Detects user attempts to override system or tool behavior.

- Privacy Awareness: Flags possible PII in inputs and outputs.



NOTE

Guardrail enforcement applies to **llm_input** and **llm_output** touchpoints only. Tool-level guardrails are not included in this release.

Llama Stack Connectors

Llama Stack Connectors provide a high-level abstraction for AI registries such as MCP. Platform Engineers can register connectors by using a **connector_id**, and AI Engineers can consume pre-registered connectors without managing complex configurations. This feature simplifies the workflow for AI engineers by abstracting away infrastructure concerns.

Conversations API

The Conversations API enables multi-turn, context-aware chats by managing message history, tool outputs, and conversation state. Developers can use this API to build AI applications with memory, moving beyond simple stateless requests to create persistent, intelligent interactions.

OpenAI-compatible annotations for search and responses in Llama Stack

Starting with OpenShift AI 3.3, Llama Stack provides OpenAI-compatible grounding and citation annotations for search-backed responses as a Technology Preview feature.
This enhancement enables retrieval-augmented generation (RAG) applications to trace generated responses back to source documents by using the same annotation schemas returned by OpenAI Search and Responses APIs. The feature supports document source attribution and preserves citation metadata in API responses, allowing existing OpenAI client applications to consume citation information without code changes.

This capability improves transparency, auditability, and explainability for enterprise RAG workloads, and serves as a foundation for future advanced tracing and observability features in Llama Stack. For more information, see OpenAI-compatible file citation annotations.

The Llama Stack Operator available on multi-architecture clusters

The Llama Stack Operator is deployable on multi-architecture clusters in OpenShift AI version 3.3 and is available by default.

Llama Stack versions in OpenShift AI 3.3

OpenShift AI 3.3.0 includes Open Data Hub Llama Stack version 0.4.2.1+rhai0, which is based on upstream Llama Stack version 0.4.2.

The Llama Stack Operator with ConfigMap driven image updates

The Llama Stack Operator in OpenShift AI 3.3 now offers ConfigMap driven image updates for LlamaStackDistribution resources. This allows you to patch security or bug fixes without new operator versions. To enable this feature, update your ConfigMap with the following parameters:

```
image-overrides: |
  starter-gpu: registry.redhat.io/rhoai/odh-llama-stack-core-rhel9:v3.3
  starter: registry.redhat.io/rhoai/odh-llama-stack-core-rhel9:v3.3
```

Using the **starter-gpu** and **starter** distributions names as the key allows the operator to apply these overrides automatically.

To update the Llama Stack Distributions image for all **starter** distributions, run the following command:

```
$ kubectl patch configmap llama-stack-operator-config -n llama-stack-k8s-operator-system --type merge -p '{"data":{"image-overrides":"starter: quay.io/opendatahub/llama-stack:latest"}}'
```

This allows the LlamaStackDistribution resources to restart with the new image.

Model-as-a-Service (MaaS) integration

This feature is available as a Technology Preview.
OpenShift AI now includes Model-as-a-Service (MaaS) to address resource consumption and governance challenges associated with serving large language models (LLMs).

MaaS provides centralized control over model access and resource usage by exposing models through managed API endpoints, allowing administrators to enforce consumption policies across teams.

This Technology Preview introduces the following capabilities:

- Policy and quota management
- Authentication and authorization
- Usage tracking
- User management
- Zero-Touch setup through Red Hat OpenShift AI operator
  For more information, see Governing LLM access with models-as-a-service .

MLServer ServingRuntime for KServe

The MLServer serving runtime for KServe is now available as a technology preview feature in Red Hat OpenShift AI. You can use this runtime to deploy models trained on structured data, such as classical machine learning models. You can deploy models directly without converting them to ONNX format, which simplifies the deployment process and improves performance.
This feature provides support for the following common machine learning frameworks:

- scikit-learn
- XGBoost
- LightGBM
  For more information, see Deploying models using the mlserver runtime and Supported configurations.

pgvector support as a remote vector store provider in Llama Stack

Starting with OpenShift AI 3.2, you can use PostgreSQL with the pgvector extension as a remote vector store provider for the Llama Stack **vector_store** endpoint as a Technology Preview feature.
This enhancement enables vector storage backed by PostgreSQL, providing durable and transactional persistence for vector embeddings. For more information, see Llama Stack API provider support and Deploying a PostgreSQL instance with pgvector.

Llama Stack versions in OpenShift AI 3.2

OpenShift AI 3.2.0 uses the Open Data Hub Llama Stack version 0.3.5+rhai0 in the Llama Stack Distribution, which is based on the upstream Llama Stack version 0.3.5.

Llama Stack servers now require installation of the PostgreSQL Operator

In OpenShift AI 3.2, the PostgreSQL Operator is now required to deploy a Llama Stack server. For more information, see the *Deploying a Llama Stack server* documentation.

Enabling high availability on Llama Stack

Llama Stack servers can be configured to remain operational in the event of a single point of failure as a Technology Preview feature. You can enable PostgreSQL high-availability settings in your **LlamaStackDistribution** custom resource. For more information, see the *Enabling high availability on Llama Stack (Optional)* documentation.

Custom embeddings on Llama Stack

OpenShift AI 3.2 allows you to customize your embedding models as a Technology Preview feature. In the version of Llama Stack shipped in OpenShift AI 3.2, vLLM controls embeddings by default. You can update the **VLLM_EMBEDDING_URL** environment variable in your **LlamaStackDistribution** custom resource to enable embeddings, or you can use custom embeddings providers. For example:

```
- name: ENABLE_SENTENCE_TRANSFORMERS
  value: "true"
- name: EMBEDDING_PROVIDER
  value: "sentence-transformers"
```

NVIDIA NeMo Guardrails

You can use NVIDIA NeMo Guardrails as a Technology Preview feature to add guardrails and safety controls to your deployed models in Red Hat OpenShift AI. NeMo Guardrails provides a framework for controlling conversations with large language models, enabling you to define a variety of rails, such as sensitive data detection, content filtering, or custom validation rules.

Stop button for chatbot in Generative AI Studio

You can interrupt the chatbot as it is composing a response to a prompt. In the Playground, after you send a prompt, the **Send** button in the chat input field changes to a **Stop** button. Click it if you want to interrupt the model's response, for example, when the response takes longer than you anticipated or if you notice that you made an error in your prompt. The chatbot posts "You stopped this message" to confirm your stop request.

Kubeflow Trainer v2

Kubeflow Trainer v2 is now available as a Technology Preview feature in OpenShift AI 3.2.
Kubeflow Trainer v2 is the next generation of distributed training for OpenShift AI, replacing the Kubeflow Training Operator v1 (KFTOv1). This Kubernetes-native solution simplifies how data scientists and ML engineers run PyTorch training workloads at scale using a unified TrainJob API and Python SDK.

This Technology Preview release introduces the following capabilities:

- Simplified job definitions using TrainJob and TrainingRuntime resources
- Python SDK for programmatic job creation and management
- A new web-based user interface for inspecting and interacting with training jobs

- Real-time progress tracking with visibility into training steps, epochs, and metrics
- Smart checkpoint management with automatic preservation during pod preemption or termination
- Pausing and resuming train jobs
- Resource-aware scheduling via native integration with Red Hat build of Kueue
  Users of the deprecated Kubeflow Training Operator v1 (KFTOv1) should migrate their workloads to Kubeflow Trainer v2 before KFTOv1 is removed. For guidance and more details, see the migration guide.

  For more information about Kubeflow Trainer v2 features and usage, see the Kubeflow Trainer v2 documentation.

TrustyAI–Llama Stack integration for safety, guardrails, and evaluation

You can now use the Guardrails Orchestrator from TrustyAI with Llama Stack as a Technology Preview feature.
This integration enables built-in detection and evaluation workflows to support AI safety and content moderation. When TrustyAI is enabled and the FMS Orchestrator and detectors are configured, no manual setup is required.

To activate this feature, set the following field in the **DataScienceCluster** custom resource for the OpenShift AI Operator: **spec.llamastackoperator.managementState: Managed**

For more information, see the TrustyAI FMS Provider on GitHub: TrustyAI FMS Provider.

AI Available Assets page for deployed models and MCP servers

A new **AI Available Assets** page enables AI engineers and application developers to view and consume deployed AI resources within their projects.
This enhancement introduces a filterable UI that lists available models and Model Context Protocol (MCP) servers in the selected project, allowing users with appropriate permissions to identify accessible endpoints and integrate them directly into the AI Playground or other applications.

Generative AI Playground for model testing and evaluation

The Generative AI (GenAI) Playground introduces a unified, interactive experience within the OpenShift AI dashboard for experimenting with foundation and custom models.
Users can test prompts, compare models, and evaluate Retrieval-Augmented Generation (RAG) workflows by uploading documents and chatting with their content. The GenAI Playground also supports integration with approved Model Context Protocol (MCP) servers and enables export of prompts and agent configurations as runnable code for continued iteration in local IDEs.

Chat context is preserved within each session, providing a suitable environment for prompt engineering and model experimentation.

Support for air-gapped Llama Stack deployments

You can now install and operate Llama Stack and RAG/Agentic components in fully disconnected (air-gapped) OpenShift AI environments.
This enhancement enables secure deployment of Llama Stack features without internet access, allowing organizations to use AI capabilities while maintaining compliance with strict network security policies.

Feature Store integration with Workbenches and new user access capabilities

This feature is available as a Technology Preview.
The Feature Store is now integrated with OpenShift AI, data science projects, and workbenches. This integration also introduces centrally managed, role-based access control (RBAC) capabilities for improved governance.

These enhancements provide two key capabilities:

- Feature development within the workbench environment.
- Administrator-controlled user access.
  This update simplifies and accelerates feature discovery and consumption for data scientists while allowing platform teams to maintain full control over infrastructure and feature access.

Feature Store user interface

The **Feature Store** component now includes a web-based user interface (UI).
You can use the UI to view registered Feature Store objects and their relationships, such as features, data sources, entities, and feature services.

To enable the UI, edit your **FeatureStore** custom resource (CR) instance. When you save the change, the Feature Store Operator starts the UI container and creates an OpenShift route for access.

For more information, see *Setting up the Feature Store user interface for initial use* .

IBM Spyre AI Accelerator model serving support on x86 platforms

Model serving with the IBM Spyre AI Accelerator is now available as a Technology Preview feature for x86 platforms. The IBM Spyre Operator automates installation and integrates the device plugin, secondary scheduler, and monitoring. For more information, see the IBM Spyre Operator catalog entry.

Build Generative AI Apps with Llama Stack on OpenShift AI

With this release, the Llama Stack Technology Preview feature enables Retrieval-Augmented Generation (RAG) and agentic workflows for building next-generation generative AI applications. It supports remote inference, built-in embeddings, and vector database operations. It also integrates with providers like TrustyAI's provider for safety and Trusty AI's LM-Eval provider for evaluation. This preview includes tools, components, and guidance for enabling the Llama Stack Operator, interacting with the RAG Tool, and automating PDF ingestion and keyword search capabilities to enhance document discovery.

Centralized platform observability

Centralized platform observability, including metrics, traces, and built-in alerts, is available as a Technology Preview feature. This solution introduces a dedicated, pre-configured observability stack for OpenShift AI that allows cluster administrators to perform the following actions:

- View platform metrics (Prometheus) and distributed traces (Tempo) for OpenShift AI components and workloads.
- Manage a set of built-in alerts (alertmanager) that cover critical component health and performance issues.

- Export platform and workload metrics to external 3rd party observability tools by editing the **DataScienceClusterInitialization** (DSCI) custom resource.
  You can enable this feature by integrating with the Cluster Observability Operator, Red Hat build of OpenTelemetry, and Tempo Operator. For more information, see Monitoring and observability. For more information, see Managing observability.

Support for Llama Stack Distribution version 0.3.0

The Llama Stack Distribution now includes version 0.3.0 as a Technology Preview feature.
This update introduces several enhancements, including expanded support for retrieval-augmented generation (RAG) pipelines, improved evaluation provider integration, and updated APIs for agent and vector store management. It also provides compatibility updates aligned with recent OpenAI API extensions and infrastructure optimizations for distributed inference.

The previously supported version was 0.2.22.

Support for Kubernetes Event-driven Autoscaling (KEDA)

OpenShift AI now supports Kubernetes Event-driven Autoscaling (KEDA) in its KServe RawDeployment mode. This Technology Preview feature enables metrics-based autoscaling for inference services, allowing for more efficient management of accelerator resources, reduced operational costs, and improved performance for your inference services.
To set up autoscaling for your inference service in KServe RawDeployment mode, you need to install and configure the OpenShift Custom Metrics Autoscaler (CMA), which is based on KEDA.

For more information about this feature, see: Configuring metrics-based autoscaling.

LM-Eval model evaluation UI feature

TrustyAI now offers a user-friendly UI for LM-Eval model evaluations as Technology Preview. This feature allows you to input evaluation parameters for a given model and returns an evaluation-results page, all from the UI.

Support for creating and managing Ray Jobs with the CodeFlare SDK

You can now create and manage Ray Jobs on Ray Clusters directly through the CodeFlare SDK.
This enhancement aligns the CodeFlare SDK workflow with the KubernetesFlow Training Operator (KFTO) model, where a job is created, run, and completed automatically. This enhancement simplifies manual cluster management by preventing Ray Clusters from remaining active after job completion.

Support for direct authentication with an OIDC identity provider

Direct authentication with an OpenID Connect (OIDC) identity provider is now available as a Technology Preview feature.
This enhancement centralizes OpenShift AI service authentication through the Gateway API, providing a secure, scalable, and manageable authentication model. You can configure the Gateway API with your external OIDC provider by using the **GatewayConfig** custom resource.

Custom flow estimator for Synthetic Data Generation pipelines

You can now use a custom flow estimator for synthetic data generation (SDG) pipelines.
For supported and compatible tagged SDG teacher models, the estimator helps you evaluate a chosen teacher model, custom flow, and supported hardware on a sample dataset before running full workloads.

Llama Stack support and optimization for single node OpenShift (SNO)

Llama Stack core can now deploy and run efficiently on single node OpenShift (SNO).
This enhancement optimizes component startup and resource usage so that Llama Stack can operate reliably in single-node cluster environments.

FAISS vector storage integration

You can now use the FAISS (Facebook AI Similarity Search) library as an inline vector store in OpenShift AI.
FAISS is an open-source framework for high-performance vector search and clustering, optimized for dense numerical embeddings with both CPU and GPU support. When enabled with an embedded SQLite backend in the Llama Stack Distribution, FAISS stores embeddings locally within the container, removing the need for an external vector database service.

New Feature Store component

You can now install and manage Feature Store as a configurable component in OpenShift AI. Based on the open-source Feast project, Feature Store acts as a bridge between ML models and data, enabling consistent and scalable feature management across the ML lifecycle.
This Technology Preview release introduces the following capabilities:

- Centralized feature repository for consistent feature reuse
- Python SDK and CLI for programmatic and command-line interactions to define, manage, and retrieve features for ML models
- Feature definition and management
- Support for a wide range of data sources
- Data ingestion via feature materialization
- Feature retrieval for both online model inference and offline model training
- Role-Based Access Control (RBAC) to protect sensitive features
- Extensibility and integration with third-party data and compute providers
- Scalability to meet enterprise ML needs
- Searchable feature catalog
- Data lineage tracking for enhanced observability
  For configuration details, see Configuring Feature Store.

FIPS support for Llama Stack and RAG deployments

You can now deploy Llama Stack and RAG or agentic solutions in regulated environments that require FIPS compliance.
This enhancement provides FIPS-certified and compatible deployment patterns to help organizations meet strict regulatory and certification requirements for AI workloads.

Validated sdg-hub notebooks for Red Hat AI Platform

Validated **sdg_hub** example notebooks are now available to provide a notebook-driven user experience in OpenShift AI 3.0.
These notebooks support multiple Red Hat platforms and enable customization through SDG pipelines. They include examples for the following use cases:

- Knowledge and skills tuning, including annotated examples for fine-tuning models.
- Synthetic data generation with reasoning traces to customize reasoning models.
- Custom SDG pipelines that demonstrate using default blocks and creating new blocks for specialized workflows.

RAGAS evaluation provider for Llama Stack (inline and remote)

You can now use the Retrieval-Augmented Generation Assessment (RAGAS) evaluation provider to measure the quality and reliability of RAG systems in OpenShift AI.
RAGAS provides metrics for retrieval quality, answer relevance, and factual consistency, helping you identify issues and optimize RAG pipeline configurations.

The integration with the Llama Stack evaluation API supports two deployment modes:

- Inline provider: Runs RAGAS evaluation directly within the Llama Stack server process.
- Remote provider: Runs RAGAS evaluation as distributed jobs using OpenShift AI pipelines. The RAGAS evaluation provider is now included in the Llama Stack distribution.

Enable targeted deployment of workbenches to specific worker nodes in Red Hat OpenShift AI Dashboard using node selectors

The hardware profiles feature enables users to target specific worker nodes for workbenches or model-serving workloads. It allows users to target specific accelerator types or CPU-only nodes. This feature replaces the current accelerator profiles feature and container size selector field, offering a broader set of capabilities for targeting different hardware configurations. While accelerator profiles, taints, and tolerations provide some capabilities for matching workloads to hardware, they do not ensure that workloads land on specific nodes, especially if some nodes lack the appropriate taints.

The hardware profiles feature supports both accelerator and CPU-only configurations, along with node selectors, to enhance targeting capabilities for specific worker nodes. Administrators can configure hardware profiles in the settings menu. Users can select the enabled profiles using the UI for workbenches, model serving, and AI pipelines where applicable.

RStudio Server workbench image

With the RStudio Server workbench image, you can access the RStudio IDE, an integrated development environment for R. The R programming language is used for statistical computing and graphics to support data analysis and predictions.
To use the RStudio Server workbench image, you must first build it by creating a secret and triggering the **BuildConfig**, and then enable it in the OpenShift AI UI by editing the **rstudio-rhel9** image stream. For more information, see Building the RStudio Server workbench images .

IMPORTANT

Disclaimer: Red Hat supports managing workbenches in OpenShift AI. However, Red Hat does not provide support for the RStudio software. RStudio Server is available through rstudio.org and is subject to their licensing terms. You should review their licensing terms before you use this sample workbench.

CUDA - RStudio Server workbench image

With the CUDA - RStudio Server workbench image, you can access the RStudio IDE and NVIDIA CUDA Toolkit. The RStudio IDE is an integrated development environment for the R programming language for statistical computing and graphics. With the NVIDIA CUDA toolkit, you can enhance your work by using GPU-accelerated libraries and optimization tools.
To use the CUDA - RStudio Server workbench image, you must first build it by creating a secret and triggering the **BuildConfig**, and then enable it in the OpenShift AI UI by editing the **rstudio-rhel9** image stream. For more information, see Building the RStudio Server workbench images .

IMPORTANT

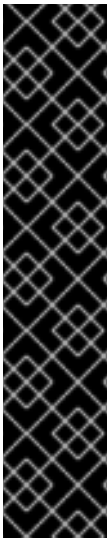
Disclaimer: Red Hat supports managing workbenches in OpenShift AI. However, Red Hat does not provide support for the RStudio software. RStudio Server is available through rstudio.org and is subject to their licensing terms. You should review their licensing terms before you use this sample workbench.

The CUDA - RStudio Server workbench image contains NVIDIA CUDA technology. CUDA licensing information is available in the CUDA Toolkit documentation. You should review their licensing terms before you use this sample workbench.

Support for multinode deployment of very large models

Serving models over multiple graphical processing unit (GPU) nodes when using a single-model serving runtime is now available as a Technology Preview feature. Deploy your models across multiple GPU nodes to improve efficiency when deploying large models such as large language models (LLMs). For more information, see Deploying models by using multiple GPU nodes .

CHAPTER 4. DEVELOPER PREVIEW FEATURES



IMPORTANT

This section describes Developer Preview features in Red Hat OpenShift AI 3.4. Developer Preview features are not supported by Red Hat in any way and are not functionally complete or production-ready. Do not use Developer Preview features for production or business-critical workloads. Developer Preview features provide early access to functionality in advance of possible inclusion in a Red Hat product offering. Customers can use these features to test functionality and provide feedback during the development process. Developer Preview features might not have any documentation, are subject to change or removal at any time, and have received limited testing. Red Hat might provide ways to submit feedback on Developer Preview features without an associated SLA.

For more information about the support scope of Red Hat Developer Preview features, see Developer Preview Support Scope.

4.1. 3.4 GA DEVELOPER PREVIEW FEATURES

AgentCard support for post-deployment agent discovery

You can now discover deployed agents and their capabilities through the AgentCard custom resource. When you deploy an agent as a Kubernetes Deployment or StatefulSet with the **kagenti.io/type: agent** label and a protocol label such as **protocol.kagenti.io/a2a**, the platform automatically creates an AgentCard that advertises the agent's capabilities, endpoints, and supported protocols. This enables machine-readable agent-to-agent discovery for multi-agent workflows. For more information, see Kagenti Operator Repository.

Agent deploy and runtime management

You can now manage the runtime concerns of deployed agents using the AgentRuntime custom resource. Create an AgentRuntime that references your agent's Deployment or StatefulSet via **spec.targetRef**, and the operator automatically injects authentication and identity sidecars into agent pods. This includes an Envoy-based AuthBridge proxy for inbound JWT validation and outbound token exchange, SPIFFE-based workload identity via a spiffe-helper sidecar, and per-agent OpenTelemetry trace configuration. Agents labeled with **kagenti.io/type: agent** are injected by default, but you can opt out with **kagenti.io/inject: disabled**. The AgentRuntime also supports per-workload overrides for SPIFFE trust domains, OTEL collector endpoints, and trace sampling rates. For more information, see Kagenti Operator Repository.

Distributed Tracing for Distributed Inference with llm-d

Platform Operators can trace distributed inference with llm-d requests end-to-end across service boundaries using OpenTelemetry-compatible distributed tracing. Traces correlate latency and errors across the full request path, from gateway through router-scheduler to model servers.

Batch inference compatible with OpenAI batch APIs in Distributed Inference with llm-d

Platform Operators can submit large request volumes asynchronously through the OpenAI-compatible **/v1/batches** API and retrieve results without maintaining an active connection. Batch workloads run at lower priority than real-time traffic and are scheduled during periods of low cluster activity. For more information, see Batch Gateway and Deploy Batch Gateway on Kubernetes .

Existing vector stores available as RAG knowledge sources in Gen AI Studio Playground

You can surface previously-created vector stores as retrieval-augmented generation (RAG) knowledge sources in the Gen AI Studio Playground. Platform engineers define vector stores

through ConfigMaps, and AI engineers can select them as knowledge sources when chatting with models in the Playground. This enables rapid RAG experimentation without writing code or re-ingesting data.
This release includes the following capabilities:

- Platform engineers can declare vector stores through ConfigMaps with connection details, collection names, and optional metadata
- Available vector stores appear under RAG / Knowledge Sources in the Playground
- Users can enable or disable a vector store per chat session
- Queries are routed through Llama Stack RAG primitives

NOTE

This feature does not include document upload, ingestion, chunking, indexing, or vector store lifecycle management. Only read-only query and retrieval of pre-existing vector data is supported.

Interact with Red Hat OpenShift AI using MCP clients

Red Hat OpenShift AI provides an MCP (Model Context Protocol) server that enables MCP-compatible clients to interact with your environment through natural-language conversations. When you describe your goals, the MCP server recommends the optimal model from your model registry by matching your intent against benchmarks such as MMLU and HumanEval, with cost comparisons. You can also manage data science projects, create workbenches, and monitor pipeline runs.
When you are ready to deploy, the MCP server generates three production-ready Kubernetes manifests for model serving, auto-scaling, and observability. The MCP server works with AI coding assistants such as Claude Code, OpenCode, and Gemini CLI. For more information, see RHOAI MCP server.

Tool calling metadata on model cards in the model catalog

Red Hat OpenShift AI now displays tool callin, also known as function calling, configuration metadata directly in the model catalog. This update ensures that users have immediate access to the specific CLI arguments and chat templates required to successfully deploy models for agentic or tool-use workflows.
Previously, users had to manually identify which arguments were necessary for a model to interact with external APIs or functions. Validated models in the catalog now include this technical metadata as a single source of truth, accessible directly on the model card.

Key features of this update include:

- Metadata visibility: Tool-calling commands and configuration requirements are now rendered in the YAML frontmatter of the model card. Users can view and copy these parameters directly from the UI to ensure deployment accuracy.
- New Tool calling filter: A **Tool calling** task filter has been added to the left-hand navigation pane in the model catalog. Users can quickly filter the catalog for models specifically validated by the Red Hat Model Validation team for tool-calling capabilities.
- Validated parameters: For supported models, the catalog now provides specific CLI requirements, such as **--enable-auto-tool-choice**, **--tool-call-parser**, and the correct **chat_template** in the metadata on the tool calling section of the model card in the model

catalog.

To identify models that support external tool integration:

1. In the OpenShift AI dashboard, from the left navigation menu, click **AI hub → Catalog**.
2. In the left filter pane, under the **Task** section, select **Tool calling**.
   The catalog refreshes to show only models that have been validated for tool-use compatibility.

To view and copy the specific arguments required for a tool-calling deployment:

1. Select a model from the **Tool calling** filtered list, for example, **Ministral-3-14B-Instruct-2512**.
2. Click the model to view its model card and scroll to the model details frontmatter section to locate the tool-calling metadata.
3. Copy the validated CLI arguments for use in your model-serving configuration in the additional arguments text box.

MCP Catalog for enterprise management of MCP servers

The MCP Catalog provides a centralized experience for discovering, deploying, and experimenting with Model Context Protocol (MCP) servers in Red Hat OpenShift AI. AI Operators and Platform Engineers can browse available MCP servers in a catalog UI, view descriptive metadata about each server's capabilities and tools, and deploy MCP servers into their namespace directly from the catalog. After deployment, a platform engineer can register deployed MCP servers in the gen AI studio configuration, making them available in the playground for interactive experimentation.
The MCP Catalog ships pre-loaded with MCP servers from Red Hat, technology partners, and the open source community. These servers can be deployed directly from the catalog without sourcing container images or configuring transport manually. Some additional prerequisite steps, such as mirroring images may be required for disconnected deployments or configuring server-specific credentials.

- Red Hat MCP servers:
  - Red Hat OpenShift - Cluster management and troubleshooting
  - Red Hat Ansible Automation Platform - Playbook execution and configuration orchestration
  - Red Hat Insights - platform intelligence and remediation recommendations.
- Technology partner MCP servers:
  - Confluent Cloud - Kafka and Flink streaming
  - EDB Postgres AI - database queries and schema management
  - HashiCorp Terraform - infrastructure as code
  - Microsoft Azure - cloud resource management
  - Dynatrace - performance monitoring and troubleshooting.
- Other MCP servers: MongoDB (document collections and RAG workflows) and MariaDB (relational database connectivity).

The MCP Catalog relies on the following pre-requisite upstream community components:

- mcp-lifecycle-operator: A Kubernetes operator that provides a declarative API to deploy, manage, and roll out MCP servers. When an MCPServer custom resource is created, the operator automatically provisions the required Deployments, Services, and cluster-internal URLs for service discovery. The MCP lifecycle operator must be installed on the cluster before deploying MCP servers from the catalog. For more information, see kubernetes-sigs/mcp-lifecycle-operator on GitHub.

- mcp-gateway: A Kubernetes-native gateway that provides a unified runtime endpoint for accessing deployed MCP servers. The MCP Gateway aggregates tools from multiple registered MCP servers behind a single endpoint, enabling centralized access control and routing. For more information, see the mcp-gateway/docs/guides/quick-start.md on GitHub.

The deploy action in the catalog UI is gated on the presence of the MCP lifecycle operator. If the operator is not installed, the deployment option is not available.

4.2. 3.4 EA2 DEVELOPER PREVIEW FEATURES

Automate machine learning model training with AutoML

AutoML is available as a Developer Preview feature in Red Hat OpenShift AI 3.4. You can use AutoML to automatically train and compare machine learning models for your tabular data. AutoML evaluates multiple models and ranks the results in a leaderboard. For the top-performing models, AutoML generates trained model artifacts and a notebook that you can use to deploy the best model. For more information, see AutoML on GitHub.

Automate RAG optimization with AutoRAG

AutoRAG is available as a Developer Preview feature in Red Hat OpenShift AI 3.4. You can use AutoRAG to find optimal RAG configurations for your documents and use cases. AutoRAG evaluates multiple configurations and ranks the results in a leaderboard. For the top-performing configurations, AutoRAG generates Jupyter notebooks that you can use to deploy your RAG application. For more information, see AutoRAG on GitHub.

Core Evaluation Stack control plane

The Evaluation Stack control plane provides an API REST routing and orchestration layer for AI evaluation, benchmarking, and profiling backends on OpenShift AI. AI engineers can deploy and manage a comprehensive evaluation platform that supports multiple frameworks and execution modes through a unified interface.

The Evaluation Stack control plane includes the following capabilities:

- REST API endpoints for programmatic evaluation triggering from web UI interfaces and other components

- Built-in support for LM Evaluation Harness, RAGAS, Garak, and GuideLLM evaluation frameworks

- Custom framework integration by using container images or Python package specifications for Kubeflow Pipelines

- Evaluation results tracked in MLflow with a standardized schema

- Evaluation job monitoring and progress tracking through Kubernetes constructs

- Concurrent evaluation jobs with resource isolation
- Support for air-gapped and disconnected environments
- Installable Python package for local development

4.3. 3.4 EA1 DEVELOPER PREVIEW FEATURES

Automatic MLflow experiment creation in EvalHub

The EvalHub service automatically creates an MLflow experiment when you specify **experiment.name** in the evaluation job request. If the experiment creation fails due to missing MLflow configuration, authentication issues, or other problems, the job request returns an error.

Kubeflow Spark Operator for distributed data processing

The Kubeflow Spark Operator is now available in OpenShift AI as a Developer Preview. This feature provides Apache Spark integration with OpenShift AI, enabling the full lifecycle of Spark applications on Kubernetes. This enables unified orchestration and monitoring of large-scale data processing and preparation Spark jobs alongside ML training and inference workflows.
To enable the Kubeflow Spark Operator, navigate to your **dsc.yaml** CR and update the **kubeflowsparkoperator** parameter to the **Managed** state.

This feature introduces the following capabilities:

- Integration with the OpenShift AI distributed workloads ecosystem.
- **SparkApplication** custom resources (CRs) for defining Spark jobs.
- Automatic submission, monitoring and restart of Spark applications with configuration retry policies.
- Pod customization via mutating webhooks, supporting ConfigMaps, volumes, and affinity rules.

Run evaluations for TrustyAI-Llama Stack using LM-Eval

You can now run evaluations using LM-Eval on Llama Stack with TrustyAI as a Developer Preview feature, using the built-in LM-Eval component and advanced content moderation tools. To use this feature, ensure TrustyAI is enabled, the FMS Orchestrator and detectors are set up, and KServe RawDeployment mode is in use for full compatibility if needed. There is no manual set up required. Then, in the **DataScienceCluster** custom resource for the Red Hat OpenShift AI Operator, set the **spec.llamastackoperator.managementState** field to **Managed**.

For more information, see the following resources on GitHub:

- Trusty AI Eval Provider
- LLama Stack Operator

LLM Compressor integration

LLM Compressor capabilities are now available in Red Hat OpenShift AI as a Developer Preview feature. A new workbench image with the **llm-compressor** library and a corresponding data science pipelines runtime image make it easier to compress and optimize your large language models (LLMs) for efficient deployment with vLLM. For more information, see **llm-compressor** in GitHub.

You can use LLM Compressor capabilities in two ways:

- Use a Jupyter notebook with the workbench image available at Red Hat Quay.io: **opendatahub** / **llmcompressor-workbench**.
  For an example Jupyter notebook, see **examples/llmcompressor/workbench_example.ipynb** in the **red-hat-ai-examples** repository.

- Run a data science pipeline that executes model compression as a batch process with the runtime image available at Red Hat Quay.io: **opendatahub** / **llmcompressor-pipeline-runtime**.
  For an example pipeline, see **examples/llmcompressor/oneshot_pipeline.py** in the **red-hat-ai-examples** repository.

MLflow integration

OpenShift AI now includes a Developer Preview of MLflow. MLflow uses Kubernetes namespaces (OpenShift projects) as workspaces to provide logical isolation of experiments, registered models, and prompts. MLflow uses Kubernetes role-based access control (RBAC) to authorize API requests. For more information about enabling and using MLflow in OpenShift AI, see the Configuring MLflow in OpenShift AI (Developer Preview) Knowledgebase article.

AI Available Assets integration with Model-as-a-Service (MaaS)

This feature is available as a Developer Preview.
You can now access and consume Model-as-a-Service (MaaS) models directly from the **AI Available Assets** page in the GenAI Studio.

Administrators can configure a MaaS by enabling the toggle in the **Model Deployments** page. When a model is marked as a service, it becomes global and visible across all projects in the cluster.

Additional fields added to Model Deployments for AI Available Assets integration

This feature is available as a Developer Preview.
Administrators can now add metadata to models during deployment so that they are automatically listed on the **AI Available Assets** page.

The following table describes the new metadata fields that streamline the process of making models discoverable and consumable by other teams:

Field name	Field type	Description
Use Case	Free-form text	Describes the model's primary purpose, for example, "Customer Churn Prediction" or "Image Classification for Product Catalog."
Description	Free-form text	Provides more detailed context and functionality notes for the model.
Add to AI Assets	Checkbox	When enabled, automatically publishes the model and its metadata to the **AI Available Assets** page.

Compatibility of Llama Stack remote providers and SDK with MCP HTTP streaming protocol

This feature is available as a Developer Preview.
Llama Stack remote providers and the SDK are now compatible with the Model Context Protocol (MCP) HTTP streaming protocol.

This enhancement enables developers to build fully stateless MCP servers, simplify deployment on standard Llama Stack infrastructure (including serverless environments), and improve scalability. It also prepares for future enhancements such as connection resumption and provides a smooth transition away from Server-Sent Events (SSE).

Packaging of ITS Hub dependencies to the Red Hat–maintained Python index

This feature is available as a Developer Preview.
All Inference Time Scaling (ITS) runtime dependencies are now packaged in the Red Hat-maintained Python index, allowing Red Hat AI and OpenShift AI customers to install **its_hub** and its dependencies directly by using **pip**.

This enhancement enables users to build custom inference images with ITS algorithms focused on improving model accuracy at inference time without requiring model retraining, such as:

- Particle filtering
- Best-of-N
- Beam search
- Self-consistency
- Verifier or PRM-guided search
  For more information, see the ITS Hub on GitHub.

Dynamic hardware-aware continual training strategy

Static hardware profile support is now available to help users select training methods, models, and hyperparameters based on VRAM requirements and reference benchmarks. This approach ensures predictable and reliable training workflows without dynamic hardware discovery.
The following components are included:

- **API Memory Estimator**: Accepts model, training method, dataset metadata, and assumed hyperparameters as input and returns an estimated VRAM requirement for the training job. Delivered as an API within Training Hub.
- **Reference Profiles and Benchmarks**: Provides end-to-end training time benchmarks for OpenShift AI Innovation (OSFT) and Performance Team (LAB SFT) baselines, delivered as static tables and documentation in Training Hub.
- **Hyperparameter Guidance**: Publishes safe starting ranges for key hyperparameters such as learning rate, batch size, epochs, and LoRA rank. Integrated into example notebooks maintained by the AI Innovation team.

IMPORTANT

Hardware discovery is not included in this release. Only static reference tables and guidance are provided; automated GPU or CPU detection is not yet supported.

Human-in-the-Loop (HIL) functionality in the Llama Stack agent

Human-in-the-Loop (HIL) functionality has been added to the Llama Stack agent to allow users to approve unread tool calls before execution.
This enhancement includes the following capabilities:

- Users can approve or reject unread tool calls through the responses API.
- Configuration options specify which tool calls require HIL approval.
- Tool calls pause until user approval is received for HIL-enabled tools.
- Tool calls that do not require HIL continue to run without interruption.

CHAPTER 5. SUPPORT REMOVALS

This section describes major changes in support for user-facing features in Red Hat OpenShift AI. For information about OpenShift AI supported software platforms, components, and dependencies, see the Supported Configurations for 3.x Knowledgebase article.

5.1. DEPRECATED

Deprecated default group creation for model registry

Starting with OpenShift AI 3.4, the default group creation performed by the OpenShift AI Operator when a model registry is created is deprecated. This default group will be removed in a future release of OpenShift AI. The OpenShift administrator will then be responsible for creating this group after the model registry is created, if needed in their workflow.

5.1.1. Ray-based multi-node vLLM template

In Red Hat OpenShift AI 3.3, the Ray-based multi-node vLLM template remains available as a Technology Preview. Starting with Red Hat OpenShift AI 3.4, Ray will be removed from the vLLM multi-node ServingRuntime and multi-node inference will rely on native vLLM multiprocessing support.

Customers can continue using the Ray-based multi-node template in 3.3 (Tech Preview).

5.1.2. Training images and ClusterTrainingRuntimes for Kubeflow Training Operator v1

The Kubeflow Training Operator (v1) is deprecated starting OpenShift AI 2.25 and is scheduled to be removed. This deprecation is part of our transition to Kubeflow Trainer v2, which delivers enhanced capabilities and improved functionality.

New and/or updated container images and associated ClusterTrainingRuntimes will be released for Kubeflow Trainer v2 in Red Hat OpenShift AI 3.4, and the existing runtimes and container images will be deprecated. Guidance for updating to the new runtimes and images will be provided in the Red Hat OpenShift AI 3.4 release.

The list of images being deprecated in Red Hat OpenShift AI 3.4 is:

- registry.redhat.io/rhoai/odh-training-cuda121-torch24-py311-rhel9
- registry.redhat.io/rhoai/odh-training-cuda124-torch25-py311-rhel9
- registry.redhat.io/rhoai/odh-training-cuda128-torch28-py312-rhel9
- registry.redhat.io/rhoai/odh-training-cuda128-torch29-py312-rhel9
- registry.redhat.io/rhoai/odh-training-rocm62-torch24-py311-rhel9
- registry.redhat.io/rhoai/odh-training-rocm62-torch25-py311-rhel9
- registry.redhat.io/rhoai/odh-training-rocm64-torch28-py312-rhel9
- registry.redhat.io/rhoai/odh-training-rocm64-torch29-py312-rhel9

The list of ClusterTrainingRuntimes being deprecated in Red Hat OpenShift AI 3.4 is:

- training-hub-th05-cuda128-torch29-py312

- torch-distributed-cuda128-torch29-py312
- torch-distributed-rocm64-torch29-py312
  For the list of supported configuration see Red Hat OpenShift AI: Supported Configurations for 3.x .

5.1.3. Deprecated SQLite as a production metadata store for Llama Stack

Starting with OpenShift AI 3.2, SQLite is deprecated for use as a metadata store in production Llama Stack deployments. PostgreSQL is required for production-grade environments to ensure adequate performance, concurrency, and scalability. SQLite remains available for local development and testing only and must be explicitly configured. This includes configurations that define SQLite backends such as **kv-sqlite** or **sql-sqlite** in the Llama Stack storage configuration. SQLite is not intended for production workloads.

5.1.4. Deprecated annotation format for Connection Secrets::

Starting with OpenShift AI 3.0, the **opendatahub.io/connection-type-ref** annotation format for creating Connection Secrets is deprecated.

For all new Connection Secrets, use the **opendatahub.io/connection-type-protocol** annotation instead. While both formats are currently supported, **connection-type-protocol** takes precedence and should be used for future compatibility.

5.1.5. Deprecated Kubeflow Training operator v1

The Kubeflow Training Operator (v1) is deprecated starting OpenShift AI 2.25 and is planned to be removed in a future release. This deprecation is part of our transition to Kubeflow Trainer v2, which delivers enhanced capabilities and improved functionality.

5.1.6. Deprecated TrustyAI service CRD v1alpha1

Starting with OpenShift AI 2.25, the **v1apha1** version is deprecated and planned for removal in an upcoming release. You must update the TrustyAI Operator to version **v1** to receive future Operator updates.

5.1.7. Deprecated KServe Serverless deployment mode

Starting with OpenShift AI 2.25, The KServe Serverless deployment mode is deprecated. You can continue to deploy models by migrating to the KServe RawDeployment mode. If you are upgrading to Red Hat OpenShift AI 3.0, all workloads that use the retired Serverless or ModelMesh modes must be migrated before upgrading.

5.1.8. Deprecated model registry API v1alpha1

Starting with OpenShift AI 2.24, the model registry API version **v1alpha1** is deprecated and will be removed in a future release of OpenShift AI. The latest model registry API version is **v1beta1**.

5.1.9. Multi-model serving platform (ModelMesh)

Starting with OpenShift AI version 2.19, the multi-model serving platform based on ModelMesh is deprecated. You can continue to deploy models on the multi-model serving platform, but it is recommended that you migrate to the single-model serving platform.

For more information or for help on using the single-model serving platform, contact your account manager.

5.1.10. Accelerator Profiles and legacy Container Size selector deprecated

Starting with OpenShift AI 3.0, Accelerator Profiles and the **Container Size** selector for workbenches are deprecated.

+ These features are replaced by the more flexible and unified Hardware Profiles capability.

5.1.11. Deprecated OpenVINO Model Server (OVMS) plugin

The CUDA plugin for the OpenVINO Model Server (OVMS) is now deprecated and will no longer be available in future releases of OpenShift AI.

5.1.12. OpenShift AI dashboard user management moved from **OdhDashboardConfig** to **Auth** resource

Previously, cluster administrators used the **groupsConfig** option in the **OdhDashboardConfig** resource to manage the OpenShift groups (both administrators and non-administrators) that can access the OpenShift AI dashboard. Starting with OpenShift AI 2.17, this functionality has moved to the **Auth** resource. If you have workflows (such as GitOps workflows) that interact with **OdhDashboardConfig**, you must update them to reference the **Auth** resource instead.

Table 5.1. Updated configurations

Resource	2.16 and earlier	2.17 and later versions
apiVersion	**opendatahub.io/v1alpha**	**services.platform.opendatahub.io/v1alpha1**
kind	**OdhDashboardConfig**	**Auth**
name	**odh-dashboard-config**	**auth**
Admin groups	**spec.groupsConfig.adminGroups**	**spec.adminGroups**
User groups	**spec.groupsConfig.allowedGroups**	**spec.allowedGroups**

5.1.13. Deprecated cluster configuration parameters

When using the CodeFlare SDK to run distributed workloads in Red Hat OpenShift AI, the following parameters in the Ray cluster configuration are now deprecated and should be replaced with the new parameters as indicated.

Deprecated parameter	Replaced by
head_cpus	**head_cpu_requests**, **head_cpu_limits**

Deprecated parameter	Replaced by
head_memory	**head_memory_requests**, **head_memory_limits**
min_cpus	**worker_cpu_requests**
max_cpus	**worker_cpu_limits**
min_memory	**worker_memory_requests**
max_memory	**worker_memory_limits**
head_gpus	**head_extended_resource_requests**
num_gpus	**worker_extended_resource_requests**

You can also use the new **extended_resource_mapping** and **overwrite_default_resource_mapping** parameters, as appropriate. For more information about these new parameters, see the CodeFlare SDK documentation (external).

5.2. REMOVED FUNCTIONALITY

tf2onnx package removed from TensorFlow images

The **tf2onnx** package has been removed from TensorFlow workbench and runtime images. This package, which converts TensorFlow models to ONNX format, was incompatible with Keras 3 (used in TensorFlow 2.16+) and had irreconcilable dependency conflicts with **protobuf** versions required by **onnx**, **tensorflow**, and **feast**. The upstream project has been abandoned since January 2024.
If you require TensorFlow to ONNX conversion, see RHAIENG-1632 for alternative approaches.

Caikit-NLP component removed

The **caikit-nlp** component has been formally deprecated and removed from OpenShift AI 3.0.
This runtime is no longer included or supported in OpenShift AI. Users should migrate any dependent workloads to supported model serving runtimes.

TGIS component removed

The TGIS component, which was deprecated in OpenShift AI 2.19, has been removed in OpenShift AI 3.0.
TGIS continued to be supported through the OpenShift AI 2.16 Extended Update Support (EUS) lifecycle, which ended in June 2025.

Starting with this release, TGIS is no longer available or supported. Users should migrate their model serving workloads to supported runtimes such as Caikit or Caikit-TGIS.

AppWrapper Controller removed

The AppWrapper controller has been removed from OpenShift AI as part of the broader CodeFlare Operator removal process.
This change eliminates redundant functionality and reduces maintenance overhead and architectural complexity.

5.2.1. CodeFlare Operator removed

Starting with OpenShift AI 3.0, the CodeFlare Operator has been removed.

+ The functionality previously provided by the CodeFlare Operator is now included in the KubeRay Operator, which provides equivalent capabilities such as mTLS, network isolation, and authentication.

LAB-tuning feature removed

Starting with OpenShift AI 3.0, the LAB-tuning feature has been removed.
Users who previously relied on LAB-tuning for large language model customization should migrate to alternative fine-tuning or model customization methods.

Embedded Kueue component removed

The embedded Kueue component, which was deprecated in OpenShift AI 2.24, has been removed in OpenShift AI 3.0.
OpenShift AI now uses the Red Hat Build of the Kueue Operator to provide enhanced workload scheduling across distributed training, workbench, and model serving workloads.

The embedded Kueue component is not supported in any Extended Update Support (EUS) release.

Removal of DataSciencePipelinesApplication v1alpha1 API version

The **v1alpha1** API version of the **DataSciencePipelinesApplication** custom resource (**datasciencepipelinesapplications.opendatahub.io/v1alpha1**) has been removed.
OpenShift AI now uses the stable **v1** API version (**datasciencepipelinesapplications.opendatahub.io/v1**).

You must update any existing manifests or automation to reference the **v1** API version to ensure compatibility with OpenShift AI 3.0 and later.

5.2.2. Microsoft SQL Server command-line tool removal

Starting with OpenShift AI 2.24, the Microsoft SQL Server command-line tools (sqlcmd, bcp) have been removed from workbenches. You can no longer manage Microsoft SQL Server using the preinstalled command-line client.

5.2.3. Model registry ML Metadata (MLMD) server removal

Starting with OpenShift AI 2.23, the ML Metadata (MLMD) server has been removed from the model registry component. The model registry now interacts directly with the underlying database by using the existing model registry API and database schema. This change simplifies the overall architecture and ensures the long-term maintainability and efficiency of the model registry by transitioning from the **ml-metadata** component to direct database access within the model registry itself.

If you see the following error for your model registry deployment, this means that your database schema migration has failed:

> error: error connecting to datastore: Dirty database version {version}. Fix and force version.

You can fix this issue by manually changing the database from a dirty state to 0 before traffic can be routed to the pod. Perform the following steps:

1. Find the name of your model registry database pod as follows:
   kubectl get pods -n <your-namespace> | grep model-registry-db

   Replace **<your-namespace>** with the namespace where your model registry is deployed.

2. Use **kubectl exec** to run the query on the model registry database pod as follows:
   kubectl exec -n <your-namespace> <your-db-pod-name> -c mysql -- mysql -u root -p"$MYSQL_ROOT_PASSWORD" -e "USE <your-db-name>; UPDATE schema_migrations SET dirty = 0;"

   Replace **<your-namespace>** with your model registry namespace and **<your-db-pod-name>** with the pod name that you found in the previous step. Replace **<your-db-name>** with your model registry database name.

   This will reset the dirty state in the database, allowing the model registry to start correctly.

5.2.4. Embedded subscription channel not used in some versions

For OpenShift AI 2.8 to 2.20 and 2.22 to 3.4, the **embedded** subscription channel is not used. You cannot select the **embedded** channel for a new installation of the Operator for those versions. For more information about subscription channels, see Installing the Red Hat OpenShift AI Operator.

5.2.5. Anaconda removal

Anaconda is an open source distribution of the Python and R programming languages. Starting with OpenShift AI version 2.18, Anaconda is no longer included in OpenShift AI, and Anaconda resources are no longer supported or managed by OpenShift AI.

If you previously installed Anaconda from OpenShift AI, a cluster administrator must complete the following steps from the OpenShift command-line interface to remove the Anaconda-related artifacts:

1. Remove the secret that contains your Anaconda password:
   oc delete secret -n redhat-ods-applications anaconda-ce-access

2. Remove the **ConfigMap** for the Anaconda validation cronjob:
   oc delete configmap -n redhat-ods-applications anaconda-ce-validation-result

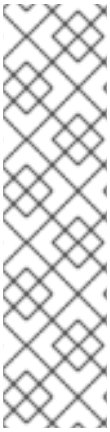
3. Remove the Anaconda image stream:
   oc delete imagestream -n redhat-ods-applications s2i-minimal-notebook-anaconda

4. Remove the Anaconda job that validated the downloading of images:
   oc delete job -n redhat-ods-applications anaconda-ce-periodic-validator-job-custom-run

5. Remove any pods related to Anaconda cronjob runs:
   oc get pods n redhat-ods-applications --no-headers=true | awk '/anaconda-ce-periodic-validator-job-custom-run*/'

5.2.6. Pipeline logs for Python scripts running in Elyra pipelines are no longer stored in S3

Logs are no longer stored in S3-compatible storage for Python scripts which are running in Elyra pipelines. From OpenShift AI version 2.11, you can view these logs in the pipeline log viewer in the OpenShift AI dashboard.



NOTE

For this change to take effect, you must use the Elyra runtime images provided in workbench images at version 2024.1 or later.

If you have an older workbench image version, update the **Version selection** field to a compatible workbench image version, for example, 2024.1, as described in Updating a project workbench.

Updating your workbench image version will clear any existing runtime image selections for your pipeline. After you have updated your workbench version, open your workbench IDE and update the properties of your pipeline to select a runtime image.

5.2.7. Beta subscription channel no longer used

Starting with OpenShift AI 2.5, the **beta** subscription channel is no longer used. You can no longer select the **beta** channel for a new installation of the Operator. For more information about subscription channels, see Installing the Red Hat OpenShift AI Operator .

5.2.8. HabanaAI workbench image removal

Support for the HabanaAI 1.10 workbench image has been removed. New installations of OpenShift AI from version 2.14 do not include the HabanaAI workbench image. However, if you upgrade OpenShift AI from a previous version, the HabanaAI workbench image remains available, and existing HabanaAI workbench images continue to function.

CHAPTER 6. RESOLVED ISSUES

The following notable issues are resolved in Red Hat OpenShift AI 3.4 EA1. Security updates, bug fixes, and enhancements for Red Hat OpenShift AI 3.4 EA1 are released as asynchronous errata. All OpenShift AI errata advisories are published on the Red Hat Customer Portal.

6.1. ISSUES RESOLVED IN RED HAT OPENSHIFT AI 3.4 EA1

RHOAIENG-46420 - Inference failures with vLLM when using the temperature parameter on IBM Z

Before this update, on IBM Z platforms, vLLM inference failed when the temperature parameter was explicitly set in inference requests. Any request that included the temperature field caused the vLLM process to terminate, and the serving pod entered a **CrashLoopBackOff** state. This issue occurred only when the temperature parameter was present in the request.

This issue is resolved. You can use the temperature parameter in vLLM inference requests on IBM Z platforms.

6.2. ISSUES RESOLVED IN RED HAT OPENSHIFT AI 3.3

RHOAIENG-24545 - Runtime images are not present in workbench after first start

Before this update, the list of runtime images was not properly populated for the first running workbench instance in the namespace. As a result, no image was shown for selection in the Elyra pipeline editor and required a workaround to populate the runtime image list.

The list of runtime images is now properly populated even for the first running workbench instance in the namespace without needing any extra workaround. Elyra now contains the required runtime image list in the editor as expected from the first workbench start. This issue has been resolved.

6.3. ISSUES RESOLVED IN RED HAT OPENSHIFT AI 3.2

RHOAIENG-31071 - LM-Eval evaluations using Parquet datasets fail on IBM Z (s390x)

Before this update, Apache Arrow's Parquet implementation contained endianness-specific code that was incompatible with big-endian IBM Z (**s390x**) architecture, causing byte-order mismatches when reading Parquet-formatted datasets. This resulted in LM-Eval evaluation tasks using datasets in Parquet format failing on **s390x** systems with parsing errors. A workaround applied compatibility patches to Apache Arrow and built a custom version specifically for **s390x** to support proper Parquet encoding/decoding.

RHOAIENG-38579 - Cannot stop models served with the Distributed Inference Server runtime

Before this update, you could not stop models served with the **Distributed Inference Server with llm-d** runtime from the OpenShift AI dashboard. This issue has been resolved.

RHOAIENG-38180 - Unable to send requests to Feature Store using the Feast SDK from workbench

Before this update, Feast was missing certificates and a service when running the default configuration, which prevented you from sending requests to your Feature Store by using the Feast SDK.

This issue has been resolved.

RHOAIENG-41588 - Standard openshift-container-platform route support added for dashboard access

Before this update, the transition to using Gateway API for Red Hat OpenShift AI version 3.0 required load balancer configuration. This configuration requirement caused usability issues and led to deployment delays for users of baremetal and cloud infrastructures. This issue has been resolved. The Gateway API now supports Cluster IP mode and standard openshift-container-platform route configuration in addition to the load balancer configuration option, simplifying dashboard access for the users.

For more information, see Configurable Ingress Mode for RHOAI 3.2 on Bare Metal, OpenStack and Private Clouds.

RHOAIENG-44616 - Inferencing with granite-3b model fails on IBM Power

Before this update, inference services for the **granite-3b-code-instruct-2k** model were created successfully. However, when a chat completion request was sent, it failed with an **Internal server error**. This issue is now resolved.

RHOAIENG-37686 - Metrics not displayed on the Dashboard due to image name mismatch in runtime detection logic

Previously, metrics were not displayed on the OpenShift AI dashboard because digest-based image names were not correctly recognized by the runtime detection system. This issue affected all **InferenceService** deployments in OpenShift AI 2.25 and later. This issue has been resolved.

RHOAIENG-37492 - Dashboard console link not accessible on IBM Power in 3.0.0

Previously, on private cloud deployments running on IBM Power, the OpenShift AI dashboard link was not visible in the OpenShift console when the dashboard was enabled in the **DataScienceCluster** configuration. As a result, users could not access the dashboard through the console without manually creating a route. This issue has been resolved.

RHOAIENG-1152 - Basic workbench creation process fails for users who have never logged in to the dashboard

This issue is now obsolete as of OpenShift AI 3.0. The basic workbench creation process has been updated, and this behavior no longer occurs.

RHOAIENG-9418 - Elyra raises error when you use parameters in uppercase

Previously, Elyra raised an error when you tried to run a pipeline that used parameters in uppercase. This issue is now resolved.

RHOAIENG-30493 - Error creating a workbench in a Kueue-enabled project

Previously, when using the dashboard to create a workbench in a Kueue-enabled project, the creation failed if Kueue was disabled on the cluster or if the selected hardware profile was not associated with a LocalQueue. In this case, the required LocalQueue could not be referenced, the admission webhook validation failed, and an error message was shown. This issue has been resolved.

RHOAIENG-32942 - Elyra requires unsupported filters on the REST API when pipeline store is Kubernetes

Before this update, when the pipeline store was configured to use Kubernetes, Elyra required equality (**eq**) filters that were not supported by the REST API. Only substring filters were supported in this mode. As a result, pipelines created and submitted through Elyra from a workbench could not run successfully.

This issue has been resolved.

RHOAIENG-32897 - **Pipelines defined with the Kubernetes API and invalid `platformSpec` do not appear in the UI or run**

Before this update, when a pipeline version defined with the Kubernetes API included an empty or invalid **spec.platformSpec** field (for example, **{}** or missing the **kubernetes** key), the system misidentified the field as the pipeline specification. As a result, the REST API omitted the **pipelineSpec**, which prevented the pipeline version from being displayed in the UI and from running. This issue is now resolved.

RHOAIENG-31386 - **Error deploying an Inference Service with `authenticationRef`**

Before this update, when deploying an InferenceService with **authenticationRef** under external metrics, the **authenticationRef** field was removed. This issue is now resolved.

RHOAIENG-33914 - **LM-Eval Tier2 task test failures**

Previously, there could be failures with LM-Eval Tier2 task tests because the Massive Multitask Language Understanding Symbol Replacement (MMLUSR) tasks were broken. This issue is resolved witih the latest version of the **trustyai-service-operator**.

RHOAIENG-35532 - **Unable to deploy models with `HardwareProfiles` and GPU**

Before this update, the HardwareProfile to use GPU for model deployment had stopped working. The issue is now resolved.

RHOAIENG-4570 - **Existing Argo Workflows installation conflicts with install or upgrade**

Previously, installing or upgrading OpenShift AI on a cluster that already included an existing Argo Workflows instance could cause conflicts with the embedded Argo components deployed by Data Science Pipelines. This issue has been resolved. You can now configure OpenShift AI to use an existing Argo Workflows instance, enabling clusters that already run Argo Workflows to integrate with Data Science Pipelines without conflicts.

RHOAIENG-35623 - **Model deployment fails when using hardware profiles**

Previously, model deployments that used hardware profiles failed because the Red Hat OpenShift AI Operator did not inject the **tolerations**, **nodeSelector**, or **identifiers** from the hardware profile into the underlying **InferenceService** when manually creating **InferenceService** resources. As a result, the model deployment pods could not be scheduled to suitable nodes and the deployment fails to enter a ready state. This issue is now resolved.

CHAPTER 7. KNOWN ISSUES

This section describes known issues in Red Hat OpenShift AI 3.4 EA1, 3.4 EA2, and 3.4 GA, and any known methods of working around these issues.

7.1. ISSUES DISCOVERED AT VERSION 3.4 GA

RHOAIENG-37916 - Deployed llm-d model shows failed in UI Models deployed using the "{llm-d}" will initially show a **Status of Failed** in the OpenShift AI Dashboard. To get more information on actual status, use the OpenShift Console to follow the status of pods in the project. When the model is fully ready, the OpenShift AI Dashboard will display a status of **Started**" Workaround:: Wait until the status changes, or consult the pod statuses.

RHOAIENG-60940 - BUG: NemoGuardrails RBAC ClusterRoles missing Kubernetes aggregation labels cause 403 for non-cluster-admin users

When testing NemoGuardrails with a non-cluster-admin user, a 403 Forbidden error is returned: bq. User "X" cannot get resource "nemoguardrails" in API group trustyai.opendatahub.io in namespace "Y".

The nemoguardrail-viewer-role and nemoguardrail-editor-role ClusterRoles on the operator side are missing the standard Kubernetes RBAC aggregation labels (aggregate-to-view, aggregate-to-edit, aggregate-to-admin). Without these labels, regular namespace users do not inherit the NemoGuardrails permissions through the default aggregated ClusterRoles, resulting in access denied errors.

Workaround

Cluster admins can create ClusterRoleBindings that apply the nemoguardrail-editor-role or nemoguardrail-viewer-role to users that require edit or view permissions to NeMo Guardrail resources:

```
$ oc create clusterrolebinding <binding-name> \ --clusterrole=nemoguardrail-editor-role \ --user=<username>
```

```
$ oc create clusterrolebinding <binding-name> \ --clusterrole=nemoguardrail-viewer-role \ --user=<username>
```

RHOAIENG-60855 - Upgrade error: Llama Stack Operator produces invalid Deployment when storage is configured

When upgrading OpenShift AI from 3.3 to 3.4, the Llama Stack Operator can fail to reconcile an existing LlamaStackDistribution custom resource that includes a storage specification, for example **storage.size: 2Gi**. Due to an upgrade-strategy change, the operator may generate an invalid Deployment that specifies both spec.strategy.type: Recreate and spec.strategy.rollingUpdate, which Kubernetes rejects with an error similar to: **Deployment.apps "llama-stack-distribution-upgrade" is invalid: spec.strategy.rollingUpdate: Forbidden: may not be specified when strategy 'type' is 'Recreate'**

Workaround

Delete the affected Deployment so that the operator recreates it with a valid strategy:

```
oc delete deployment <cr-name> -n <namespace>
```

Replace <cr-name> with the name of the LlamaStackDistribution custom resource and <namespace> with its namespace. Llama stack operator will recreate deployment and new pod will work as expected.

RHOAIENG-60292 - MLflow does not automatically run database migrations after upgrade

There are database migration changes that are automatically applied after upgrade.

Workaround

It is recommended to bring down the MLflow replicas to 1 on the MLflow CR, the **spec.replicas** parameter, during upgrade.

RHOAIENG-58969 - precise-prefix-cache-scorer returns score of zero due to PodIdentifier key format mismatch

In OpenShift AI 3.4, the precise prefix cache scorer was updated to identify routing endpoints using a combination of the IP address and port. Previous versions relied solely on the IP address. If the vLLM configuration is not updated to provide the port, the scorer cannot match cache entries to the correct endpoints. As a result, traffic routing ignores prefix cache locality entirely and falls back to standard load balancing, which can reduce performance efficiency.

Workaround

Update the vLLM arguments in your configuration to include the port number in the KV events topic. Modify the topic format from the older format (kv@${POD_IP}@<model>) to include the port. For example, **kv@${POD_IP}:8000@<model>**. The precise scorer will correctly identify cache locations, successfully restoring prefix cache locality and routing requests to the pods with the most relevant cached data.

RHOAIENG-59950 - Search space preparation fails when too many models are provided

User starts AutoRAG experiment with more than 3 embedding models or more than 2 generation models. Consequence: **search_space_preparation** component runs models pre-selection and produces incorrect **search_space_prep_report** that cannot be properly parsed in the next component: **rag_templates_optimization**. User sees:

> AutoRAG fails with: SearchSpaceValueError: Missing required parameters in the search space: {'embedding_model', 'foundation_model'}

Workaround

User may start the experiment with up to 2 foundation models and up to 3 embedding models.

7.2. ISSUES DISCOVERED AT VERSION 3.4 EA2

RHOAIENG-60236 - Workbench images and external accelerator runtimes are not part of Designed for FIPS

Red Hat OpenShift AI workbench images and external accelerator runtimes are not part of Designed for FIPS. While the core platform components support FIPS-enabled clusters, workbench images and external accelerator runtimes have not been designed or tested for FIPS compliance.

Workaround

None.

RHOAIENG-58765 - Distributed Inference with llm-d prefill and decode disaggregation fails on FIPS-enabled clusters

Using Distributed Inference with llm-d prefill and decode disaggregation for LLM deployments on FIPS-enabled clusters causes the routing sidecar pod to enter a crash loop, preventing the LLM deployment from functioning. This issue is caused by a runtime image introduced in the 3.4 EA2 release that is not FIPS-compatible.

Workaround

Do not use prefill and decode disaggregation with Distributed Inference with llm-d in Red Hat OpenShift AI 3.4 EA2 on FIPS-enabled clusters. Other features continue to work correctly on FIPS-enabled clusters.

RHOAIENG-3816 - Encrypted PDF uploads to Llama Stack vector stores fail on FIPS-enabled clusters

On FIPS-enabled clusters, registering certain encrypted PDF files into Llama Stack vector stores fails due to a limitation in the underlying PDF parsing library. The library uses an MD5-based digest as part of its encryption handling, which is not allowed in FIPS mode and triggers an UnsupportedDigestmodError: [digital envelope routines] unsupported error during file processing. Due to this, on FIPS-enabled clusters, affected encrypted PDFs cannot be uploaded into Llama Stack vector stores. As a result, these documents are not indexed and are unavailable to retrieval-augmented generation (RAG) workflows, which may lead to incomplete or missing answers when those documents are expected to be part of the context.

Workaround

Use non-encrypted PDF files when ingesting documents into Llama Stack vector stores on FIPS-enabled clusters, or re-export or convert the original document to a non-encrypted PDF before uploading it to the vector store. Until the underlying PDF library is updated to use FIPS-compliant cryptographic primitives, encrypted PDFs that rely on disallowed digests in FIPS mode will continue to fail to upload to Llama Stack vector stores on FIPS-enabled clusters.

RHOAIENG-57224 - ROCm universal image training produces NaN on MI300X due to torch aotriton 0.11.1 regression

ROCm universal training image (th06) produces NaN values on MI300X due to aotriton 0.11.1 regression in AIPCC-built PyTorch wheel.

Workaround

Use th05 image or set **attn_implementation="flash_attention_2"**.

RHOIAIENG-57427 - RAG in Gen AI Playground doesn't work with default system prompt and model Qwen/Qwen3-14B-AWQ

In Gen AI Playground RAG, the default system prompt might not reliably trigger the knowledge search/tool-calling behavior for some models, so document retrieval is not performed. Due to this, questions about uploaded documents can return answers without using the vector store, resulting in incomplete/incorrect responses unless the prompt is adjusted.

Workaround

Manually edit the system prompt to explicitly instruct the model to use the knowledge search tool first for document-based/factual questions (as documented in the Gen AI Playground RAG documentation). As a result, after updating the system prompt, RAG retrieval works and the model can answer based on the uploaded document content.

RHOAIENG-54005 - Generate MaaS Token Endpoint Removed - breaks Gen AI Studio Playground

The **/v1/token** API was removed and this endpoint was merged in with the new post creation of **/v1/api-keys**. As a result, Gen AI Playground cannot generate a token on the fly for MaaS and cannot talk to MaaS Models in 3.4 EA2.

Workaround

There is no existing workaround for this known issue. As a result, there is no access to MaaS and Playground in 3.4 EA2.

RHOAIENG-48753 - Pipeline Name must be DNS-compliant to use "Store pipeline definitions in Kubernetes"

Elyra does not convert the pipeline name to a DNS-compliant name when using the default Kubernetes storage. As a consequence, if you don't use a DNS-compliant name when you start an Elyra pipeline, it gives a cryptic error "[TIP: did you mean to set *https://ds-pipeline-dspa-robert-tests.apps.test.rhoai.rh-aiservices-bu.com/pipeline* as the endpoint, take care not to include s at end]".

Workaround

Use DNS-compliant naming when running Elyra pipelines.

7.3. ISSUES DISCOVERED AT VERSION 3.4 EA1

RHOAIENG-54101 - Deployments not listed in Model Registry on IBM Z

When you deploy a model from the Model Registry on IBM Z, the deployment does not appear under the **Deployments** tab in the Model Registry.

Workaround

Access and manage the deployment from the global **Deployments** page in the OpenShift AI dashboard.

RHOAIENG-53206 - Spark driver pods fail to communicate due to RpcTimeoutException

After installing the Spark Operator, Spark executor pods cannot communicate with the driver pod because the **redhat-ods-applications** namespace defaults to a "deny-all" traffic rule. SparkApplication pods hang and fail with an **RpcTimeoutException**.

Workaround

Create a NetworkPolicy in the **redhat-ods-applications** namespace to allow communication between the pods created by the SparkApplication controller:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: spark-operator-allow-internal
spec:
  podSelector:
    matchLabels:
      sparkoperator.k8s.io/launched-by-spark-operator: "true"
  policyTypes:
    - Ingress
  ingress:
    - ports:
        - port: 7078
          protocol: TCP
```

```
    - port: 7079
      protocol: TCP
    - port: 4040
      protocol: TCP
  from:
    - podSelector: {}
    - namespaceSelector:
        matchLabels:
          network.openshift.io/policy-group: ingress
```

RHOAIENG-52130 - Workbenches with Feast integration fail to start due to missing ConfigMap

Workbenches with Feast integration enabled fail to start in OpenShift AI 3.4 EA1. Pods remain stuck in **ContainerCreating** state with the following error:

+

```
[FailedMount] [Warning] MountVolume.SetUp failed for volume "odh-feast-config"
  configmap "jupyter-nb-kube-3aadmin-feast-config" not found
```

Workaround

Restart the Feast Operator after DSC deployment completes:

```
$ kubectl rollout restart deployment/feast-operator-controller-manager -n redhat-ods-applications
```

RHOAIENG-53239 - Custom ServingRuntime required for IBM Z (s390x) vLLM Spyre deployments

When deploying models using the vLLM Spyre runtime on IBM Z (s390x) systems, the default ServingRuntime cannot be used directly for KServe-based deployments. Model deployment fails if the runtime is used without modification.

Workaround

Create a custom ServingRuntime by duplicating the **vllm-spyre-s390x-runtime** ServingRuntime and removing the **command** section from the container specification. Keep all other configuration, including environment variables, ports, and volume mounts, unchanged.
The following example shows only the affected section. Your complete ServingRuntime must include all other fields from the original template:

```
apiVersion: serving.kserve.io/v1alpha1
kind: ServingRuntime
metadata:
  name: vllm-spyre-s390x-runtime-copy
spec:
  containers:
    - name: kserve-container
      image: <image>
      # Remove the 'command' section that appears here in the original
      args:
        - --model=/mnt/models
        - --port=8000
        - --served-model-name={{.Name}}
      # ... keep all env, ports, volumeMounts from original ...
```

RHOAIENG-50523 – Unable to upload RAG documents in Gen AI Playground on disconnected clusters

On disconnected clusters, uploading documents in the Gen AI Playground RAG section fails. The progress bar never exceeds 50% because Llama Stack attempts to download the **ibm-granite/granite-embedding-125m-english** embedding model from HuggingFace, even though the model is already included in the Llama Stack Distribution image in OpenShift AI 3.3.

Workaround

Modify the LlamaStackDistribution custom resource to include the following environment variables:

```
export MY_PROJECT=my-project

oc patch llamastackdistribution lsd-genai-playground \
  -n $MY_PROJECT \
  --type='json' \
  -p='[
    {
      "op": "add",
      "path": "/spec/server/containerSpec/env/-",
      "value": {
        "name": "SENTENCE_TRANSFORMERS_HOME",
        "value": "/opt/app-root/src/.cache/huggingface/hub"
      }
    },
    {
      "op": "add",
      "path": "/spec/server/containerSpec/env/-",
      "value": {
        "name": "HF_HUB_OFFLINE",
        "value": "1"
      }
    },
    {
      "op": "add",
      "path": "/spec/server/containerSpec/env/-",
      "value": {
        "name": "TRANSFORMERS_OFFLINE",
        "value": "1"
      }
    },
    {
      "op": "add",
      "path": "/spec/server/containerSpec/env/-",
      "value": {
        "name": "HF_DATASETS_OFFLINE",
        "value": "1"
      }
    }
  ]'
```

The Llama Stack pod restarts automatically after applying this configuration.

RHAIENG-2827 – Unsecured routes created by older CodeFlare SDK versions

Existing 2.x workbenches continue to use an older version of the CodeFlare SDK when used in OpenShift AI 3.x. The older version of the SDK creates unsecured OpenShift routes on behalf of the user.

Workaround

To resolve this issue, update your workbench to the latest image provided in OpenShift AI 3.x before using CodeFlare SDK.

RHOAIENG-48867 - TrainJob fails to resume after Red Hat OpenShift AI upgrade due to immutable JobSet spec

TrainJobs that are suspended (e.g., queued by Kueue) before a Red Hat OpenShift AI upgrade cannot resume after the upgrade completes. The Trainer controller fails to update the immutable JobSet **spec.replicatedJobs** field.

Workaround

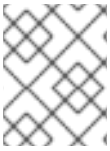
To resolve this issue, delete and recreate the affected TrainJob after the upgrade.

RHOAIENG-45142 - Dashboard URLs return 404 errors after upgrading Red Hat OpenShift AI from 2.x to 3.x

The Red Hat OpenShift AI dashboard URL subdomain changed from **rhods-dashboard-redhat-ods-applications.apps.<cluster>`to `data-science-gateway.apps.<cluster>** due to the use of Gateways in OpenShift AI version 3.x. Existing bookmarks to the dashboard using the default **rhods-dashboard-redhat-ods-applications.apps.<cluster>** format will no longer function after you upgrade to OpenShift AI version 3.0 or later. It is recommended that you update your bookmarks and any internal documentation to use the new URL format: **data-science-gateway.apps.<cluster>**.

Workaround

To resolve this issue, deploy an nginx-based redirect solution that recreates the old route name and redirects traffic to the new gateway URL. For instructions, see Dashboard URLs return 404 errors after RHOAI upgrade from 2.x to 3.x



NOTE

Cluster administrators must provide the new dashboard URL to all Red Hat OpenShift AI administrators and users. In a future release, URL redirects may be supported.

RHOAIENG-43686 - Red Hat build of Kueue 1.2 installation or upgrade fails with Kueue CRD reconciliation error

Installing Red Hat build of Kueue 1.2 or upgrading from Red Hat build of Kueue 1.1 to 1.2 fails if legacy Kueue CustomResourceDefinitions (CRDs) remain in the cluster from a previous Red Hat OpenShift AI 2.x installation. As a result, when the legacy **v1alpha1** CRDs are present, the Kueue operator cannot reconcile successfully and the Data Science Cluster (DSC) remains in a **Not Ready** state.

Workaround

To resolve this issue, delete the legacy Kueue CRDs, **cohorts.kueue.x-k8s.io/v1alpha1** or **topologies.kueue.x-k8s.io/v1alpha1** from the cluster. For detailed instructions, see Red Hat Build of Kueue 1.2 installation or upgrade fails with Kueue CRD reconciliation error.

RHOAIENG-49389 - Tier management unavailable after deleting all tiers

If you delete all service tiers from **Settings** > **Tiers**, the **Create tier** button is no longer displayed. You cannot create tiers through the dashboard until at least one tier exists. To avoid this issue, ensure at least one tier remains in the system at all times.

Workaround

Create a basic tier using the CLI, then configure its settings through the dashboard. You must have cluster administrator privileges for your OpenShift cluster to perform these steps:

1. Retrieve the **tier-to-group-mapping** ConfigMap:

```
$ oc get configmap tier-to-group-mapping redhat-ods-namespace -o yaml tier-config.yaml
```

2. Edit the ConfigMap to add a basic tier definition:

```
apiVersion: v1
  kind: ConfigMap
  metadata:
    name: tier-to-group-mapping
    namespace: redhat-ods-applications
  data:
    tiers.yaml: |
      - name: basic
        displayName: Basic Tier
        level: 0
        groups:
          - system:authenticated
```

3. Apply the updated ConfigMap:

```
$ oc apply -f tier-config.yaml
```

4. In the dashboard, navigate to **Settings → Tiers** to configure rate limits for the newly created tier.

RHOAIENG-47589 - Missing Kueue validation for TrainJob

A **TrainJob** creation without a defined Kueue **LocalQueue** passes without validation check, even when Kueue managed namespace is enabled. As a result, it is possible to create TrainJob not managed by Kueue in Kueue managed namespace.

Workaround

None.

RHOAIENG-49017 - Upgrade RAGAS provider to Llama Stack 0.4.z / 0.5.z

In order to use the Ragas provider in OpenShift AI 3.3, you must update your Llama Stack distribution to use **llama-stack-provider-ragas==0.5.4**, which works with Llama Stack >=0.4.2,<0.5.0. This version of the provider is a workaround release that is using the deprecated register endpoints as a workaround. See the full compatibility matrix for more information.

Workaround

None.

RHOAIENG-44516 - MLflow tracking server does not accept Kubernetes service account tokens

Red Hat OpenShift AI does not accept Kubernetes service accounts when you authenticate through the dashboard MLflow URL.

Workaround

To authenticate with a service account token, complete the following steps:

- Create an OpenShift Route directly to the MLflow service endpoints.
- Use the Route URL as the **MLFLOW_TRACKING_URI** when you authenticate.

CHAPTER 8. PRODUCT FEATURES

Red Hat OpenShift AI provides a rich set of features for data scientists and cluster administrators. To learn more, see Introduction to Red Hat OpenShift AI.